NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY

REPORT
Statistics

Topic:

30-DAY READMISSION PREDICTION FOR DIABETIC PATIENTS

Group 13: BUI VI ANH - 11247122
NGUYEN NGO ANH THU - 11247229
NGUYEN THAI ANH THU - 11247230
Class DS66B

Supervisor: Dr. Luong Van Thien

Ha Noi, 30th October 2026

EVALUATION OF THE GRADUATION THESIS

(For the thesis supervisor)

Name of evaluating lecturer: .....

Student's full name: ..... Student ID: .....

Thesis title: .....

.....

Select the appropriate scores for the student's presentation according to the following criteria:

Very poor (1); Poor (2); Satisfactory (3); Good (4); Excellent (5)

Integration of theory and practice (20)						
1	Clearly state the urgency and importance of the topic, the problems and hypotheses (including the objectives and their relevance), as well as the scope of application of the thesis	1	2	3	4	5
2	Update on the latest research findings (domestic/international)	1	2	3	4	5
3	Clearly and thoroughly describe the research/problem-solving methodology	1	2	3	4	5
4	Include simulation/experimental results and clearly present the results obtained	1	2	3	4	5
Ability to analyze and evaluate results (15)						
5	A clear work plan, including objectives and implementation methods, systematically based on theoretical research findings	1	2	3	4	5
6	Results are presented logically and clearly, and all results are adequately analyzed and evaluated	1	2	3	4	5
7	In the conclusion, the author identifies any differences between the results obtained and the original objectives, and provides reasoning to propose feasible future directions	1	2	3	4	5
Thesis writing skills (10)						
8	The thesis follows the prescribed format, with a logical and well-organized chapter structure (tables and figures are clear, titled, numbered, and explained or referenced; margins are consistent, and spaces are used after periods, commas, etc.); each chapter has an introduction and conclusion; references are listed and citations follow the required rules	1	2	3	4	5
9	Excellent writing skills (correct sentence structure, academic style, logical and well-supported arguments, appropriate vocabulary, etc.)	1	2	3	4	5
Scientific research achievements (5) *(select one of the three cases)*						
10a	Has a scientific paper published or accepted for publication / Won third prize or higher in the Institute-level Student Scientific Research competition / Received a scientific award (international or domestic) at third prize level or higher / Filed a patent application	5				
10b	Presented to an Institute-level panel at the Student Scientific Research conference but did not receive third prize or higher / Received an honorable mention in other national or international competitions in the relevant field (e.g., the TI contest)	2				
10c	No scientific research achievements	0				
Total score		/50				
Total score converted to a 10-point scale						

Other comments *(regarding the student's attitude and work ethic)*

.....

.....

.....

.....

.....

.....

Date: ... / ... / 20...

Reviewer

(Signature and full name)

PREFACE

Diabetes is a major public health concern worldwide, placing a considerable burden on healthcare systems due to its long-term complications and the need for continuous medical care. Hospital readmission among patients with diabetes is an important issue, as it may indicate difficulties in disease management, inadequate continuity of care, or the presence of other underlying health conditions. Predicting hospital readmission can therefore provide valuable insights into patient outcomes and support efforts to improve healthcare quality and resource allocation. These considerations highlight the importance of applying data science and machine learning techniques to identify patterns associated with hospital readmission.

In recent years, machine learning has demonstrated considerable potential in healthcare analytics, particularly in identifying complex patterns within large-scale clinical datasets. However, predicting hospital readmission remains challenging due to the complexity of patient characteristics, variations in clinical conditions, missing information, and the potential influence of multiple factors on readmission outcomes. These challenges emphasize the importance of careful data preprocessing, appropriate feature selection, and reliable model evaluation to ensure that the resulting analysis is meaningful and that the performance of predictive models is assessed appropriately.

In recognition of these challenges, this project focuses on analyzing the Diabetes 130-US Hospitals dataset, which contains hospital encounter records for patients with diabetes collected from 130 US hospitals between 1999 and 2008. The project aims to explore the characteristics of the dataset, identify relevant patterns through exploratory data analysis and statistical analysis, and develop machine learning models to investigate the prediction of hospital readmission. Particular attention is given to data cleaning, missing value handling, categorical feature processing, and feature engineering, followed by model development and evaluation. Through these stages, the project seeks to establish a systematic analytical workflow and examine the extent to which the available patient and encounter information can contribute to readmission prediction.

Through the implementation and evaluation of the proposed analytical approach, this project aims not only to apply the theoretical knowledge acquired during our studies but also to develop a deeper understanding of the practical challenges involved in healthcare data analysis. Although the project is conducted for academic purposes and

does not aim to replace professional medical judgment, we hope that it will provide useful insights into the application of data science and machine learning to healthcare-related problems.

Finally, we would like to express our sincere gratitude to our supervisor for providing the knowledge, guidance, and support necessary throughout this project. We would also like to thank all team members for their contributions, cooperation, and efforts during the development process. Given the limitations of our knowledge and the time available, this project may still contain shortcomings. We therefore welcome constructive feedback and suggestions to improve our work and deepen our understanding of the subject. We sincerely appreciate the support and guidance we have received throughout this learning experience.

DECLARATION

We, Bui Vi Anh (ID: 11247122), Nguyen Ngo Anh Thu (ID: 11247229) and Nguyen Thai Anh Thu (ID: 11247230), student of class DS66B, declare that the project "30-Day Readmission Prediction for Diabetic Patients Using Machine Learning" is our own work and was carried out under the supervision of Dr. Luong Van Thien.

The data are the public Diabetes 130-US Hospitals for Years 1999–2008 dataset from the UCI Machine Learning Repository. The source code, processing pipeline and experimental results in this report were produced by our team, and all reported figures are taken directly from our experiments without alteration. Results from other studies and public repositories are cited in the References section; we quote these figures as published and have not independently reproduced them.

We accept full responsibility for the integrity of the content of this report

Ha Noi, 30th October 2026

The Declarants

BUI VI ANH

NGUYEN NGO ANH THU

NGUYEN THAI ANH THU

TABLE OF CONTENTS

LIST OF SYMBOLS AND ABBREVIATIONS i

LIST OF FIGURES iii

LIST OF TABLES v

ABSTRACT vii

Problem Statement 1

Objectives and Contributions 2

Organization of the Report 3

1.1 Introduction to Readmission Prediction 5

1.2 Challenges 7

1.2.1 Severe class imbalance (about 11% positive) 7

1.2.2 Multiple encounters per patient and data leakage risk 8

1.2.3 Heterogeneous, high cardinality data with heavy missing values 8

1.2.4 Weak predictive signal 8

1.2.5 Operational constraints and threshold choice 9

1.2.6 Fairness across patient subgroups 10

1.3 Motivation for the proposed pipeline 10

1.4 Research objectives and methods 11

1.4.1 Objectives 11

1.4.2 Methods 12

1.5 Chapter 1 summary 12

CHAPTER 2. THEORETICAL BACKGROUND AND RELATED METHODS **13**

2.1 Problem formulation 13

2.2 Related work 14

2.2.1 Traditional machine learning methods 14
2.2.2 Ensemble learning: bagging and boosting 14
2.2.3 Existing studies on 30-day readmission prediction 15
2.3 Theoretical foundations 16
2.3.1 Gradient boosting and CatBoost 16
2.3.2 Loss function and class-imbalance handling (Logloss, class weight) 17
2.3.3 Statistical tools 18
2.4 Chapter 2 conclusion 20

CHAPTER 3. RESEARCH METHODOLOGY 21

3.1 Overall architecture of the pipeline 21
3.1.1 Design objectives 21
3.1.2 Four functional blocks (C–F) 21
3.1.3 Pipeline diagram 21
3.2 Data preprocessing and feature engineering 21
3.2.1 Data cleaning, ICD-9 grouping and feature encoding 21
3.2.2 Labeling and missing-data handling 24
3.2.3 Advanced feature engineering 25
3.2.4 Patient-level data splitting 25
3.3 Candidate models and class-imbalance handling 27
3.4 Final classifier: CatBoost 28
3.5 Probability output and decision threshold 29
3.6 Training strategy and hyperparameter optimization 29
3.7 Chapter 3 conclusion 30

CHAPTER 4. EXPERIMENTS AND EVALUATION 31

4.1 Experimental objectives 31
4.2 Experimental setup 31
4.2.1 Computing environment 31
4.2.2 Training strategy 32

4.2.3 Dataset 32
4.2.4 Exploratory data analysis 32
4.2.5 Comparison models 40
4.2.6 Evaluation metrics 40
4.3 Model evaluation results 41
4.3.1 Model selection results (CV and validation) 41
4.3.2 Final model performance on the test set 43
4.3.3 Comparison with baselines and statistical significance 47
4.3.4 Comparison with published results 49
4.4 Ablation analysis 51
4.4.1 Effect of class weighting 51
4.4.2 Effect of hyperparameter tuning 51
4.4.3 Effect of the decision threshold 52
4.5 Operational and cost-sensitive analysis 53
4.5.1 Cost ratio between false negatives and false positives 53
4.5.2 Decision-curve analysis 54
4.5.3 Recall at top-k, gain and lift 54
4.6 Error, calibration and subgroup analysis 55
4.6.1 Error analysis 55
4.6.2 Calibration 56
4.6.3 Subgroup performance 57
4.7 Chapter 4 conclusion 59

CONCLUSION **60**

General conclusion **60**

REFERENCES **63**

LIST OF SYMBOLS AND ABBREVIATIONS

	Full English Name
– Domain, Healthcare & Dataset	
A1C	Hemoglobin A1c
HbA1c	Glycated Hemoglobin (Hemoglobin A1c)
ICD-9	International Classification of Diseases, Ninth Revision
LACE	Length of stay, Acuity of admission, Comorbidity, Emergency visits
UCI	University of California, Irvine
US	United States
– Machine Learning Models, Algorithms & Workflow	
CatBoost	Categorical Boosting
DT	Decision Tree
EDA	Exploratory Data Analysis
Logloss	Logistic Loss
LR	Logistic Regression
RF	Random Forest
SHAP	SHapley Additive exPlanations
XGBoost	Extreme Gradient Boosting
– Validation, Evaluation Metrics & Statistical Analysis	
AP	Average Precision
CV	Cross-Validation
CI	Confidence Interval
ECE	Expected Calibration Error
F1	F1 Score
F2	F2 Score
MCC	Matthews Correlation Coefficient
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
TP	True Positive
FP	False Positive
TN	True Negative
FN	False Negative
NB	Net Benefit
Std	Standard Deviation
SD	Standard Deviation

Continue

(Following the List of Symbols and Abbreviations Table)

	Full English Name
Val	Validation
– Infrastructure, Data Formats & Academic Standards	
CPU	Central Processing Unit
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
DOI	Digital Object Identifier

LIST OF FIGURES

Figure 1.1 **Distribution of the binary target in the training set.** Only about 11% of encounters are readmitted within 30 days 7

Figure 3.1 Overall structure of the pipeline 21

Figure 4.1 **Distribution of the binary target in the training set.** Only about 11% of encounters are readmitted within 30 days 33

Figure 4.2 **Number of encounters by age group, split by readmission within 30 days.** 33

Figure 4.3 **Length of Hospital Stay by Readmission Outcome** 34

Figure 4.4 **Distribution of Readmission Outcomes Across Primary Diagnosis Groups** 35

Figure 4.5 **Correlation Between Numerical Clinical Variables and the Readmission Target** 36

Figure 4.6 **Readmission Rate by Number of Previous Inpatient Visits** . . 36

Figure 4.7 **Correlation Matrix of Numerical Clinical Variables** 37

Figure 4.8 **Distribution of Readmission Outcomes Across Medication-Related Variables** 38

Figure 4.9 Distributions of the five winsorized count variables before (top row) and after (bottom row) winsorization at the training-set 1st and 99th percentiles. Each variable uses the same y-axis scale in both rows. . . . 39

Figure 4.10 **Confusion matrix at the selected threshold.** 44

Figure 4.11 **ROC Curve.** 45

Figure 4.12 **Precision–Recall curve with the prevalence baseline.** 46

Figure 4.13 **Average Precision (AP) comparison across models.** 47

Figure 4.14 **Recall, precision and F1 against the threshold on the test set.** 52

Figure 4.15 **Calibration Curve of the Tuned CatBoost Model.** 57

LIST OF TABLES

Table 2.1 Protocol characteristics that determine whether results are comparable. 16

Table 3.1 Winsorization limits and capped values 23

Table 3.2 Chi square test of missingness vs target (training set) 24

Table 3.3 Engineered features. 26

Table 3.4 Candidate models. 28

Table 3.5 Hyperparameter search space for CatBoost. 30

Table 4.1 Dataset summary after the split 32

Table 4.2 Cramér's V and p-value for categorical features 39

Table 4.3 Performance comparison between naive reference policies and the tuned CatBoost model on the test set. 41

Table 4.4 Grouped 3-fold cross-validation results (fit set). 42

Table 4.5 Validation results (sorted by AP). 42

Table 4.6 Headline test performance with 95% patient-cluster bootstrap intervals (2,000 replicates). 43

Table 4.7 Confusion matrix at threshold 0.1450 (Figure 4.10). 43

Table 4.8 Trivial reference policies on the same test set. 46

Table 4.9 Candidate models on the common test set. 47

Table 4.10 Paired differences in the ranking metrics and F_1 (tuned CatBoost minus competitor). 48

Table 4.11 Paired differences in the operating-point metrics at each model's own threshold. 48

Table 4.12 Published results on the same dataset family and this work. 50

Table 4.13 Effect of class weighting on AP. 51

Table 4.14 Operating points on the test set. 52

Table 4.15 Total cost (in false-alarm units) for several cost ratios. 54

Table 4.16 Validation-set thresholds minimising empirical cost. 54

Table 4.17 Recall@k, gain and lift on the held-out test set. 55

Table 4.18 Performance by subgroup on the test set. 58

ABSTRACT

Thirty-day hospital readmission is a costly and widely monitored outcome for patients with diabetes, yet it is hard to predict from administrative data. This report develops and evaluates a machine-learning pipeline for 30-day readmission on the public Diabetes 130-US Hospitals dataset (101,766 encounters, 1999–2008; 99,343 after removing patients who died or went to hospice). The task is difficult for four reasons: only about 11% of encounters are positive, patients contribute several encounters that can leak across data splits, the categorical variables are heterogeneous and heavily missing, and the available variables carry weak signal. To address these problems, the pipeline splits data by patient, applies leakage-safe cleaning, ICD-9 grouping and utilization-based feature engineering, and compares logistic regression, decision tree, random forest and CatBoost, each with and without class weighting. Model families are compared by patient-grouped cross-validation on Average Precision (AP) and then tuned with a patient-grouped randomized search. The decision threshold is chosen on a separate validation split, and the test set is used once.

On a held-out test set of 19,802 encounters (13,998 patients, 11.2% positive), the tuned CatBoost model reaches ROC-AUC 0.677 (95% CI 0.663–0.691) and AP 0.244 (95% CI 0.221–0.268), 2.18 times the random baseline. At the validation-selected threshold of 0.145 it detects 43.1% of readmissions with 21.7% precision while flagging 22.2% of patients. It ranks patients significantly better than logistic regression, random forest and decision tree in AUC and AP after Holm correction, although the margins are small (ΔAUC about 0.01). Hyperparameter tuning gives no detectable gain over default CatBoost, and class weighting does not improve ranking.

Predicted probabilities are well calibrated (ECE 0.006, calibration slope 1.03). A cost-sensitive analysis shows the model is most useful when a missed readmission costs roughly 5 to 10 times an unnecessary intervention, saving 13–16% of expected cost against the best trivial policy. Error and subgroup analyses show that the model relies strongly on prior inpatient utilization, misses patients who look like first-time admissions, and performs near chance for patients aged 90 and over.

INTRODUCTION

Problem Statement

Diabetes is a major public health concern worldwide, requiring continuous medical management and placing a substantial burden on healthcare systems. Patients with diabetes may experience complications and require repeated hospital care, making hospital readmission an important issue in healthcare management. Understanding the factors associated with readmission can help identify patterns in patient outcomes, support the evaluation of healthcare services, and provide useful information for improving patient care and resource allocation. In this context, the application of data science and machine learning techniques to healthcare data has become increasingly important, as these methods enable researchers to analyze large volumes of patient information and investigate relationships that may not be easily identified through traditional analysis.

Despite the potential of data-driven approaches, predicting hospital readmission remains challenging due to the complexity and heterogeneity of healthcare data. Patient characteristics, diagnoses, prescribed medications, previous healthcare utilization, and hospital encounter information may all contribute to readmission outcomes. In addition, real-world healthcare datasets often contain missing values, inconsistent representations of categorical variables, and a large number of features with varying levels of relevance. These characteristics require careful data preprocessing and feature engineering before meaningful statistical analysis and predictive modeling can be conducted. Furthermore, repeated encounters involving the same patient introduce an additional challenge, as information from a single patient may appear across multiple records and potentially lead to overly optimistic evaluation results if the data are not appropriately partitioned.

To address these challenges, this project focuses on the analysis of the Diabetes 130-US Hospitals dataset, which contains hospital encounter records for patients with diabetes from 130 US hospitals between 1999 and 2008. The dataset provides information on patient demographics, admission details, diagnoses, medications, and other characteristics related to hospital encounters. The project investigates the structure and quality of the data, explores patterns associated with hospital readmission, and develops a systematic workflow for preparing the data for machine learning. Particular attention is given to missing value treatment, categorical feature processing, di-

agnosis grouping, outlier handling, and the removal of uninformative features. These steps establish a foundation for subsequent model development and evaluation, with the overall aim of investigating how patient and encounter characteristics can be used to predict hospital readmission.

Objectives and Contributions

Although machine learning offers promising approaches to healthcare prediction, the reliability of predictive models depends heavily on the quality of the input data and the validity of the evaluation procedure. The Diabetes 130-US Hospitals dataset presents several practical challenges, including missing clinical information, high-dimensional categorical features, repeated encounters from individual patients, and differences in the distribution and relevance of features. Addressing these challenges requires more than simply training a classification model. A structured analytical process is necessary to ensure that the data are appropriately prepared and that model performance is assessed under a suitable evaluation design.

Therefore, the main objective of this project is to investigate the prediction of hospital readmission among patients with diabetes through data analysis and machine learning techniques. The project aims to establish a reproducible workflow covering data understanding, preprocessing, exploratory data analysis, statistical analysis, feature engineering, model development, and performance evaluation. In particular, the workflow takes patient-level grouping into account when preparing data for model evaluation, helping reduce the risk of information leakage between training and evaluation sets caused by multiple encounters from the same patient.

The specific objectives of the project are as follows:

- **Data understanding and preprocessing:** Examine the dataset's structure, variable types, data quality, and missing values; apply appropriate cleaning and preprocessing techniques to prepare the data for further analysis.

- **Exploratory and statistical analysis:** Investigate the distributions of important variables and examine potential associations between patient and encounter characteristics and hospital readmission.

- **Feature engineering:** Transform relevant variables into more suitable representations, including grouping diagnosis codes and handling categorical features, while identifying and removing uninformative or near-constant features.

- **Machine learning and evaluation:** Develop and evaluate predictive models using an appropriate experimental workflow and relevant performance metrics to assess their ability to predict hospital readmission.

- **Reproducibility and collaboration:** Organize the data processing and modeling workflow to support consistent experiments, collaborative development, and reproducibility.

The main contributions of this project are summarized as follows:

- Establishing a structured data preprocessing pipeline for the Diabetes 130-US Hospitals dataset, addressing missing values, categorical variables, diagnosis codes, outliers, and near-constant features.

- Conducting exploratory data analysis and statistical investigations to characterize the dataset and examine patterns relevant to hospital readmission.

- Preparing feature tables and patient-level grouping information to support model development and reduce the risk of patient-level data leakage during evaluation.

- Implementing a systematic framework for model evaluation and analysis, providing a basis for comparing predictive performance and identifying potential limitations.

Through these contributions, the project aims to demonstrate the importance of data quality, appropriate preprocessing, and reliable evaluation in healthcare prediction tasks. The findings are intended to support an academic understanding of readmission prediction rather than to provide a clinically validated decision-making system.

Organization of the Report

This report is organized into four main chapters, presenting the project from the initial problem definition to the experimental results and conclusions.

Chapter 1: Overview This chapter introduces the background and motivation of the project, defines the problem of hospital readmission prediction, and presents the research objectives and main contributions. It also outlines the structure of the report.

Chapter 2: Theoretical Background and Related methods The second chapter presents the fundamental concepts underlying healthcare data analysis and hospital readmission prediction. It discusses relevant data preprocessing techniques, exploratory and statistical analysis, feature engineering, machine learning approaches, and evaluation metrics. The chapter also reviews related studies and establishes the theoretical foundation for the proposed workflow.

Chapter 3: Research methodology This chapter describes the dataset and the procedures used to prepare the data for analysis and predictive modeling. It details the data cleaning process, missing value handling, categorical feature processing, diagnosis grouping, outlier treatment, feature selection, and other preprocessing steps. The chapter also explains the experimental design, data partitioning strategy, patient-level grouping, model development procedures, and evaluation framework.

Chapter 4: Experiments and evaluation The last chapter presents the experimental setup and the results obtained from exploratory analysis, statistical investigations, and predictive modeling. It evaluates model performance using appropriate metrics, compares the results across the selected approaches, and discusses the main findings, limitations, and potential directions for future work.

Finally, the report concludes by summarizing the key findings and reflecting on the contributions and limitations of the project, together with opportunities for further research into healthcare data analysis and hospital readmission prediction.

CHAPTER 1. OVERVIEW OF 30-DAY READMISSION PREDICTION FOR DIABETIC PATIENTS

Hospital readmission is one of the most closely watched indicators of healthcare quality. When a patient returns to the hospital shortly after being discharged, it often suggests that the first stay did not fully resolve the problem, or that the follow up care after discharge was not enough. Readmissions are also expensive, which is why many health systems try to identify high risk patients before they leave the hospital. Diabetes makes this problem especially important. It is a chronic disease that affects many organ systems, and patients usually have several other conditions at the same time, so their hospital stays tend to be complicated. In this report, we build a machine learning pipeline that predicts whether a diabetic patient will be readmitted within 30 days of discharge. This chapter introduces the task, the data, and the main difficulties we face, and then explains why we designed the pipeline the way we did.

1.1 Introduction to Readmission Prediction

Hospital readmission prediction is formulated as a supervised binary classification problem. Given information about a hospital encounter, a model estimates the probability that the encounter will be followed by readmission within 30 days of discharge. The available predictors include patient demographics, admission and discharge information, diagnoses, treatment procedures, medication records, and indicators of previous healthcare utilization. The objective is to identify encounters associated with early readmission risk rather than to explain the causes of readmission or replace clinical assessment.

This study uses the Diabetes 130-US Hospitals dataset, which contains 101,766 hospital encounters involving diabetic patients across 130 US hospitals between 1999 and 2008. The original dataset comprises 50 columns covering patient characteristics, hospital utilization, clinical information, and treatment. Its target variable, readmitted, has three categories: “NO”, indicating no recorded readmission; “>30”, indicating readmission after more than 30 days; and “<30”, indicating readmission within 30 days. In the original dataset, these categories account for 54,864 encounters (53.9%), 35,545 encounters (34.9%), and 11,357 encounters (11.2%), respectively.

To align the prediction task with the study objective, the original target is converted into a binary variable named target. Encounters labeled “<30” are assigned to the positive class (1), while “NO” and “>30” are combined into the negative class (0). This definition focuses specifically on early readmission, distinguishing it from later readmission and encounters without a recorded return during the observed period. However, the resulting class distribution is imbalanced, with positive cases representing approx-

imately 11% of the modeling data. This imbalance makes it important to evaluate the model's ability to identify readmissions, rather than relying solely on overall classification accuracy.

An important characteristic of the dataset is that each row represents a hospital encounter rather than a unique patient. The same patient may therefore appear in multiple records, with encounter_id identifying an encounter and patient_nbr identifying the corresponding patient. This distinction is relevant to how the dataset is organized and used during model development.

The overall workflow begins with data understanding and exploratory data analysis (EDA), followed by data cleaning and preprocessing to handle missing or unspecified values and remove unsuitable features. Diagnosis information is grouped into broader ICD-9 categories, while feature engineering derives additional variables related to prior healthcare utilization, treatment intensity, and daily rates of laboratory procedures, medical procedures, and medication use.

Several classifiers, including Logistic Regression, Decision Tree, Random Forest, and CatBoost, are compared, along with class-weighted variants to address class imbalance. CatBoost is further tuned using 'RandomizedSearchCV' with five-fold 'StratifiedGroupKFold' cross-validation. Average Precision (AP) serves as the primary model-selection metric, while ROC-AUC provides a complementary measure of discrimination.

For the tuned CatBoost model, the decision threshold is selected by maximizing the F1-score on a patient-grouped validation set. The model is then refitted on the full training set and evaluated on the held-out test set. Performance assessment includes AP, ROC-AUC, precision, recall, specificity, F1-score, and confusion matrices, alongside analyses of threshold trade-offs, calibration, uncertainty, and subgroup performance. Patient-level cluster bootstrap methods are also applied to account for dependencies between repeated encounters from the same patient.

Overall, this study develops and evaluates a structured machine learning pipeline for predicting 30-day readmission among diabetic patients. By combining patient-aware data splitting, systematic preprocessing, feature engineering, model comparison, and threshold selection, the study aims to assess how effectively historical hospital records can identify encounters associated with early readmission. The resulting predictions are intended to support the prioritization of follow-up care, while recognizing that predictive performance alone does not establish clinical effectiveness and that the model should complement, rather than replace, clinical judgment.

1.2 Challenges

Predicting hospital readmission among diabetic patients is a challenging task, not only because of the complexity of clinical records and the diversity of patient conditions, but also because of class imbalance and the need for reliable model evaluation. Developing an effective prediction pipeline requires addressing several data-related and methodological challenges to ensure that the model can identify early readmission risk while maintaining robust and meaningful performance.

1.2.1 Severe class imbalance (about 11% positive)

The first challenge is the low proportion of positive cases. After cleaning, positive encounters account for 11.44% of the training set and 11.19% of the test set, corresponding to approximately eight negative encounters for every positive one. Consequently, a model that predicts every encounter as "not readmitted" would achieve around 89% accuracy while failing to identify any readmissions, making accuracy alone an unreliable measure of performance.

Class imbalance can also affect model training, as models trained without appropriate adjustments may favor the majority class and fail to identify positive cases effectively. To address this issue, the pipeline compares class-weighted model variants, uses Average Precision (AP) as the primary model-selection metric, and optimizes the decision threshold based on the F1-score. Recall and other classification metrics are also considered to provide a more comprehensive assessment of the model's ability to identify early readmissions.

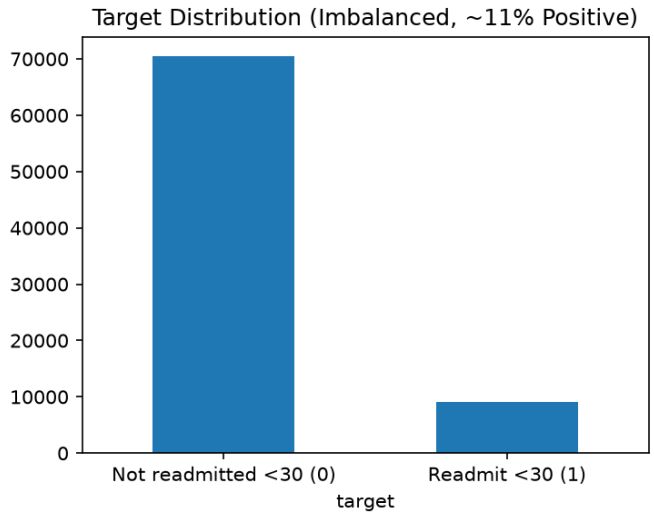


Figure 1.1 Distribution of the binary target in the training set. Only about 11% of encounters are readmitted within 30 days

1.2.2 Multiple encounters per patient and data leakage risk

The second challenge arises from the structure of the dataset. The 101,766 encounters correspond to 71,518 unique patients, giving an average of approximately 1.42 encounters per patient. Among them, 16,773 patients (around 23.5%) have more than one encounter. Since repeated encounters from the same patient may share clinical characteristics and treatment patterns, they cannot be considered fully independent. A random row-wise split could place encounters from the same patient in both the training and test sets, potentially leading to overly optimistic performance estimates. To prevent this, we split the data by patient_nbr, ensuring that no patient appears in both sets (Section 3.2.4), and use grouped cross-validation during model development.

1.2.3 Heterogeneous, high cardinality data with heavy missing values

The third difficulty is the data itself. The 50 columns mix very different types: demographic categories, administrative codes, counts of previous visits, laboratory results, and 23 columns for individual diabetes drugs. Several categorical variables have many levels. The three diagnosis columns (diag_1, diag_2, diag_3) contain 717, 749, and 790 distinct ICD 9 codes, medical_specialty has 73 levels, and payer_code has 18. Using these directly would create a very sparse representation.

Missing values are also a serious issue, and they appear in different forms. Missing entries are coded as "?" in seven columns: weight (96.9%), medical_specialty (49.1%), payer_code (39.6%), race (2.2%), diag_3 (1.4%), diag_2 (0.4%), and diag_1 (0.02%). The two laboratory columns, max_glu_serum and A1Cresult, look 94.7% and 83.3% missing when read with pandas, but the original value is the string "None", which means the test was not performed. In that case the missingness carries clinical information instead of being a data error. Finally, many medication columns are almost constant. For example, examide and citoglipton contain a single value for every encounter. This means the data needs careful, column by column handling before modelling.

1.2.4 Weak predictive signal

1. A central finding of this project is that the information available in the dataset provides only a weak signal about early readmission. This is visible at three levels.

2. Univariate association is small. In the exploratory analysis on the training set, the strongest numeric correlate of the target is the number of prior inpatient visits (Pearson correlation 0.167), followed by prior emergency visits (0.073), number of diagnoses (0.054), time in hospital (0.046), number of medications (0.041), prior outpatient visits (0.027) and number of laboratory procedures (0.025). Among categorical variables the strongest association is with discharge disposition (Cramér's

$V = 0.112$), and all remaining variables have V below 0.045; gender is not significantly associated with the target ($p = 0.516$). No single variable is a strong predictor.

3. Model discrimination is modest. The best model reaches a test ROC-AUC of 0.677 and an AP of 0.244. All eight candidate pipelines fall in a narrow ROC-AUC band between 0.645 and 0.676 on the test set, and the gap between a simple logistic regression (AP 0.218) and the best gradient-boosting model (AP 0.244) is only about 0.027 AP. In other words, a more flexible learner extracts only a little more information from the same features.

4. This is consistent with the broader literature, but the dataset-specific comparison remains provisional. Systematic reviews of hospital-readmission risk models report that models built from administrative data often achieve modest discrimination (Kansagara et al., 2011). The specific ROC-AUC range of 0.63–0.67 for studies using a clean patient-level protocol has not been independently verified from the files in this repository and should not be treated as a confirmed external benchmark. Readmission depends on factors that this dataset does not record: social support, medication adherence after discharge, functional status, laboratory values and vital signs at discharge, and the quality of outpatient follow-up.

5. The practical consequence is that evaluation must be done carefully. With a prevalence of 11.2%, the AP of a random ranking is 0.112, so the model's AP of 0.244 is a 2.18-fold improvement but still far from a reliable individual-level predictor: at the selected operating point only 21.7% of flagged encounters are true readmissions. Claims about the model must be phrased in terms of risk stratification, not diagnosis.

1.2.5 Operational constraints and threshold choice

A probability model does not by itself make a decision. To be used in a hospital, its score must be converted into a binary flag ("high risk – offer a follow-up intervention") and this conversion is constrained by operational reality.

First, intervention capacity is limited. Transition-of-care programmes can only contact or schedule a finite number of patients, so the share of discharged patients that the model flags matters as much as accuracy. Second, the two error types have unequal and context-dependent costs. A false negative is a missed readmission, with possible harm to the patient and a financial penalty for the hospital; a false positive consumes intervention resources without benefit and can cause alert fatigue if too frequent. Third, the optimal threshold depends on the class prevalence and the calibration of the model, so the conventional cut-off of 0.5 is inappropriate. A model whose probabilities average

about 0.11 will almost never output a score above 0.5, and a 0.5 cut-off would detect very few readmissions.

In this project the threshold is selected by maximising F1 on the patient-grouped validation split (15,734 encounters, 1,802 positives), giving a threshold of 0.1450. Two rules guard against optimistic bias: the threshold is never tuned on the test set, and it is recomputed from the saved validation predictions and it is computed directly from the validation predictions of the model fitted on the fit set.At this threshold, 22.2% of test encounters are flagged (4,401 of 19,802), of which 954 are true readmissions and 3,447 are false alarms, so roughly 4.6 flagged patients must be reviewed for each true readmission found. Because F1 weights precision and recall equally, it is only one possible compromise; Sections 4.4.3 and 4.5 show how the operating point changes with the threshold and with the assumed cost ratio between false negatives and false positives.

1.2.6 Fairness across patient subgroups

A model with acceptable overall performance can still perform very differently for different patient groups, and in healthcare such disparities can translate into unequal access to follow-up care (Obermeyer et al., 2019). The dataset records age, gender and race, and the diagnosis and discharge disposition define clinically meaningful groups. Two difficulties make fairness analysis hard here. First, many subgroups are small: age groups below 20 years, several races, rare diagnosis groups and most discharge-disposition codes contain fewer than 500 encounters or fewer than 50 positives, so their metrics are very noisy. Second, true prevalence differs between groups (for example 14.4% for discharge to a skilled nursing facility versus 9.0% for discharge home), which changes precision and recall even for a perfectly calibrated model.

This project therefore treats subgroup results as exploratory. Section 4.6 reports AP, ROC-AUC, recall and precision per subgroup, flags groups with fewer than 500 encounters or 50 positives as unreliable, and avoids drawing fairness or clinical conclusions from raw differences alone.

1.3 Motivation for the proposed pipeline

The difficulties listed in Section 1.2 do not have a single fix. A flexible model will not remove class imbalance by itself, and a careful split will not make a weak signal stronger. For this reason the work is organised as a pipeline in which each stage deals with one specific problem and can be checked separately. The pipeline has four components, labelled C to F in this report.

Component C is feature engineering and preprocessing. The cleaned encounter table contains only a few numerical fields, and most of them are raw counts. We add a small set of ratios and indicators that describe prior utilisation and treatment intensity,

and we scale and encode all columns inside one scikit-learn pipeline, so that no statistic is ever learned from validation or test rows.

Component D is the comparison of candidate models. Logistic regression, a decision tree, a random forest and CatBoost are each trained with and without class weights. This gives eight candidates, which are compared under the same patient-grouped cross-validation and with the same metric.

Component E is the final classifier. The best candidate, CatBoost, is tuned with a randomised search and then refitted on the full training set.

Component F turns the predicted probability into a yes/no decision. Only about 11% of encounters are positive, so the usual cut-off of 0.5 is of little use. The threshold is therefore chosen on a held-out validation split and fixed before the test set is touched.

Two design choices run through all four components. First, every split is made by patient (`patient_nbr`). One patient can contribute to several encounters, and a random row-level split would put the same person on both sides of the split and make the results look better than they are. Second, Average Precision (AP) is the main selection metric instead of accuracy or ROC-AUC, because it measures how well the model ranks the rare positive cases and it has a clear reference point: the random baseline equals the positive rate.

1.4 Research objectives and methods

1.4.1 Objectives

The study aims to build and evaluate a leakage-safe model for predicting whether a diabetic inpatient will be readmitted within 30 days of discharge. In concrete terms, the objectives are:

1. To build a reproducible pipeline in which feature engineering, preprocessing, model fitting and threshold selection are separated cleanly from the test data.

2. To compare four model families (logistic regression, decision tree, random forest and CatBoost), each with and without class weights, using patient-grouped cross-validation and AP as the selection metric.

3. To tune the selected model with a randomised hyperparameter search and to choose its decision threshold on a validation split that is disjoint from the training patients.

4. To report the performance of the final model on the untouched test set, and to show how much it improves over the random baseline and over the simpler candidates.

1.4.2 Methods

The data are the UCI "Diabetes 130-US hospitals" encounters [**?**], cleaned and split by patient into 79,541 training rows and 19,802 test rows. On top of the 31 predictors we create seven engineered features (total previous visits, two prior-visit indicators, three per-day intensity ratios and an overall treatment-intensity score), which brings the model input to 38 columns. Numerical columns are scaled with a robust scaler and categorical columns are one-hot encoded.

Inside the training set, 20% of the patients are held out as a validation split. The remaining 80% (the fit set) is used for model comparison with 3-fold `StratifiedGroupKFold` cross-validation. CatBoost is then tuned with `RandomizedSearchCV` (20 random configurations, 5 grouped folds, AP as the score). The decision threshold is set at the point that maximises F_1 on the validation predictions. Finally, the model is refitted on all training rows and evaluated once on the test set. Evaluation metrics and statistical tests are described in Chapters 2 and 4.

1.5 Chapter 1 summary

This chapter introduced the problem of predicting 30-day hospital readmission among patients with diabetes using the Diabetes 130-US Hospitals dataset. The task was formulated as a binary classification problem, with encounters followed by readmission within 30 days defined as the positive class. The chapter also highlighted six major challenges: severe class imbalance, repeated encounters from the same patient, heterogeneous data with substantial missingness, weak predictive signals, operational constraints in threshold selection, and potential disparities in model performance across patient subgroups.

To address these challenges, the study adopts a structured machine learning pipeline consisting of feature engineering and preprocessing, candidate model comparison, hyperparameter tuning, and decision threshold selection. Patient-level data splitting and grouped cross-validation are used to reduce information leakage, while Average Precision (AP) serves as the primary metric for model selection due to the low prevalence of positive cases. The final model is selected and tuned using training data, with the decision threshold determined on a separate validation set before evaluation on the untouched test set.

Overall, this chapter established the problem definition, research objectives, and methodological direction of the study. These foundations guide the theoretical discussion, detailed implementation, and experimental evaluation presented in the following chapters.

CHAPTER 2. THEORETICAL BACKGROUND AND RELATED METHODS

This chapter reviews the modelling background that the rest of the report relies on. Section 2.1 states the prediction task formally. Section 2.2 introduces the families of models that appear in the readmission literature and in our own comparison, starting from single learners and moving to ensembles. Section 2.3 then goes into the two ideas that matter most for the final model: gradient boosting as implemented in CatBoost, and the way the loss function and class weights interact with a rare positive class. The statistical tools used to judge the results are also introduced there.

2.1 Problem formulation

Unit of observation. One row corresponds to one hospital encounter, identified by encounter_id. It does not correspond to one patient. A patient (patient_nbr) may have several encounters, which is why the data must be split by patient to prevent leakage.

Input. For each encounter, the input vector x contains four groups of features: demographics (race, gender, age group), the treatment process (admission type, admission source, discharge disposition, length of stay, number of laboratory procedures, procedures, and medications, plus counts of prior outpatient, emergency, and inpatient visits), diagnoses (the primary, secondary, and tertiary diagnoses, diag_1, diag_2, and diag_3, which we group into broader ICD 9 categories), and diabetes related information (glucose serum and A1C test results, the individual diabetes medications, whether the medication was changed, and whether any diabetes medication was prescribed).

Output. The output is a binary label y. We set y = 1 if the patient is readmitted in fewer than 30 days, and y = 0 if the patient is readmitted after more than 30 days or is not readmitted. Using readmitted as the original label, y = 1 only for "<30". The model is expected to return a probability p = P(y = 1 | x), which is later converted to a class using a decision threshold.

Imbalance. The positive class makes up about 11% of the data, so the problem is highly imbalanced.

Leakage control. Because y depends on future events of the same patient, all splitting is done at the patient level. Information used for preprocessing (for example, the categories of a variable or the percentile limits for outliers) is learned from the training set only and then applied to the test set unchanged.

2.2 Related work

2.2.1 Traditional machine learning methods

Logistic regression models the log-odds of the positive class as a linear combination of the inputs. It is fast, stable and easy to explain, which is why it is the usual reference model in clinical prediction work. Its weakness is that it can only capture effects that are additive on the log-odds scale. Interactions, such as the combined effect of age and a prior inpatient stay, must be added by hand. In our setting this limitation is partly softened by the engineered features described in Section 3.2.3.

A decision tree splits the data recursively on one feature at a time, choosing the split that most reduces an impurity measure such as Gini impurity. A tree can represent non-linear effects and interactions without any manual work and can be read as a set of rules. A fully grown tree, however, fits the noise in the training data and gives unstable predictions, so its depth and leaf size are normally restricted.

A random forest [1] grows many trees on bootstrap samples and lets each split consider only a random subset of the features. Averaging the votes of these decorrelated trees lowers the variance of a single tree while keeping its ability to model non-linear patterns. Random forests usually work well with little tuning, but their probability estimates tend to be pulled toward the middle of the range, which matters when the decision threshold is far from 0.5.grows many trees on bootstrap samples and lets each split consider only a random subset of the features. Averaging the votes of these decorrelated trees lowers the variance of a single tree while keeping its ability to model non-linear patterns. Random forests usually work well with little tuning, but their probability estimates tend to be pulled toward the middle of the range, which matters when the decision threshold is far from 0.5.

Gradient-boosted trees, of which CatBoost [2] is the well-known implementation, build the trees one after another, each one correcting the errors left by the previous ones. It is often the strongest choice for tabular data with mixed feature types. We train CatBoost in this study and describe it in Section 2.3.1.

2.2.2 Ensemble learning: bagging and boosting

Ensemble methods combine several learners into one predictor. The two main strategies differ in how the learners are built and which part of the error they attack.

Bagging [3] trains independent learners on bootstrap resamples of the training set and averages their outputs. Because the learners are trained separately, bagging mainly reduces variance. It works best with unstable base learners such as deep trees, and the random forest is the standard example.

Boosting [4] trains the learners sequentially. Each new learner is fitted to what the

current ensemble still gets wrong, and its contribution is added with a small weight, the learning rate. Boosting therefore reduces bias as well as variance and can build a very accurate model from many shallow trees. The price is a higher risk of overfitting when the signal is weak or the labels are noisy, so the learning rate, the number of iterations and the tree depth have to be chosen together, usually by cross-validation.

Both strategies are relevant here. Readmission data combine many categorical fields with a few counts, and the relationship between them and the outcome is weak and noisy. A bagged ensemble is a safe, low-variance option, while boosting has more room to find structure but needs tuning. Our comparison in Chapter 4 includes one representative of each (random forest and CatBoost) so that this trade-off is measured and not assumed.

2.2.3 Existing studies on 30-day readmission prediction

The dataset. The Diabetes 130-US Hospitals dataset was assembled by Strack et al. (2014) from the Health Facts database and covers 101,766 inpatient encounters of diabetic patients in 130 hospitals between 1999 and 2008, with 50 variables including demographics, admission and discharge information, laboratory tests, diagnoses and diabetes medications. It has become a standard benchmark for readmission research and has been used with logistic regression, tree ensembles, boosting methods and neural networks.

Typical approaches. Recent studies compare a small set of familiar models. Emi-Johnson and Nkrumah (2025) evaluate logistic regression, random forest, XGBoost and a deep neural network on this dataset and report that XGBoost performs best, using SHAP values for interpretation. A study in *BMC Medical Informatics and Decision Making* (2021) builds several machine-learning classifiers for 30-day readmission of diabetic patients on a Cerner-derived data set and also identifies prior inpatient visits among the influential factors. Deep-learning papers such as EmbPred30 (Sarthak et al., 2020) and the multilayer-perceptron study of Goudjerkan and Jayabalan (2019) propose neural architectures with categorical embeddings or extensive pre-processing.

Why reported results vary so much. Reported performance on this dataset ranges from a ROC-AUC in the low 0.60s to values above 0.9. For example, EmbPred30 reports an AUROC of 97.4% and Goudjerkan and Jayabalan report an AUC of about 95%. Results of that size are far above what is achieved when the evaluation protocol prevents information from leaking between the training and test sets, and they should be interpreted with caution. A likely, though not confirmable from the abstracts alone, reason is the evaluation protocol: roughly 23% of encounters in the raw data are repeat encounters of patients who appear more than once (101,766 encounters from 71,518 patients, with 16,773 patients having several encounters), so a random split at encounter level

places the same patient in both training and test sets. Further sources of inflated performance are using features that encode the outcome (for example discharge disposition codes for death or hospice, which this project removes) and tuning thresholds on the test set. Table 2.1 summarises the protocol-related differences that matter when comparing numbers.

Table 2.1 Protocol characteristics that determine whether results are comparable.

Aspect	**Frequent practice in the literature**	**This project**
Unit of splitting	Random split of encounters (patient overlap possible)	Split by `patient_nbr`; zero patients shared (checked)
Excluded encounters	Often not stated	2,423 death/hospice encounters removed
Target	Usually readmitted < 30 days vs other	Same definition
Primary metric	Accuracy or ROC-AUC	Average Precision, ROC-AUC secondary
Threshold	Often 0.5 or tuned on test data	Max-F1 on validation, fixed before test
Uncertainty	Single point estimate	95% patient-cluster bootstrap CIs

This comparison justifies the design of the present study: rather than chasing the highest headline number, we adopt a leakage-aware protocol and report uncertainty, so that the reported AP of 0.244 and ROC-AUC of 0.677 are honest estimates of out-of-patient performance. Section 4.3.4 places these numbers next to published values and explains which comparisons are legitimate.

2.3 Theoretical foundations

2.3.1 Gradient boosting and CatBoost

Gradient boosting builds an additive model $F(x)$ by minimising a differentiable loss L one tree at a time. Starting from a constant F_0 (for binary classification, the log-odds of the positive rate), step m computes the negative gradient of the loss with respect to the current prediction for every training example, fits a small regression tree h_m to these pseudo-residuals, and updates the model:

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x) \tag{2.1}$$

Here η is the learning rate. For the logistic loss the pseudo-residual of example i is simply $y_i - p_i$, the difference between the observed label and the current predicted probability, with $p = 1/(1+\exp(-F(x)))$. After M steps the final probability is obtained by passing $F_M(x)$ through the sigmoid function.

CatBoost follows this scheme but changes several details. It uses symmetric (oblivious) trees, in which the same split condition is applied to every node of a given level. This makes the trees simple and fast to evaluate and acts as a mild regulariser. It also introduces ordered boosting and ordered target statistics, which are designed to avoid the prediction shift that appears when the target values used to build a tree are also used to encode the features of the same examples. In our experiments we keep CatBoost's default boosting scheme. The categorical columns are one-hot encoded in the preprocessing step, so CatBoost's own handling of categorical features is not used. We come back to this point in Section 3.4.

The main hyperparameters that we tune are the tree depth, the learning rate, the number of iterations and the L_2 regularisation on leaf values (`l2_leaf_reg`). A smaller learning rate usually needs more iterations, so these two are searched jointly.

2.3.2 Loss function and class-imbalance handling (Logloss, class weight)

For a binary label $y \in \{0,1\}$ and predicted probability p, the logistic loss (Logloss) averaged over N examples is:

$$L = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log(p_i) + (1-y_i)\cdot \log(1-p_i)\right] \tag{2.2}$$

Logloss is a proper scoring rule, so minimising it pushes the predicted probabilities toward the true conditional probabilities. That is a useful property, but with a positive rate near 11% the loss is dominated by negative examples. A model can reach a low Logloss by predicting small probabilities for almost everyone, and few of its outputs will exceed 0.5 even when the model ranks the positives well.

Class weights change the loss so that errors on the minority class cost more. With a weight w_c for class c, each term in the sum above is multiplied by the weight of its true class. The "balanced" heuristic sets $w_c = N/(K \cdot N_c)$, with K the number of classes. For a positive rate of 0.1143 this gives a weight of about 4.4 for positives and about 0.56 for negatives. We use `class_weight="balanced"` for logistic regression, the decision tree and the random forest, and `auto_class_weights="Balanced"` for CatBoost.

Reweighting has side effects. The predicted probabilities no longer match the real prevalence, so they cannot be read directly as risks without recalibration. The model may also give too much influence to a small number of positive examples that are partly noise. The other common remedy is to leave the loss unchanged and move the decision

threshold instead. This keeps the probabilities honest and only changes the operating point. In this study we test both options: class weights as a modelling choice (Section 4.4.1) and threshold selection as a decision rule (Section 3.5).

2.3.3 Statistical tools

We use two classical statistical tools in the exploratory analysis, the chi square test of independence and Cramér's V.

Chi square test of independence. For two categorical variables, we build a contingency table of observed counts O. Under the null hypothesis that the variables are independent, the expected count in each cell is $E = (rowtotal columntotal)/n$. The test statistic is the sum over all cells of (O minus E) squared, divided by E. A small p value (we use 0.05 as the threshold) means that the observed pattern is unlikely under independence, so we conclude that the two variables are associated. We use this test in two ways: to check whether the missingness of a variable (missing or not missing) is associated with the target, and to test the association between categorical features and the target.

Cramér's V. A drawback of the chi square test is that its value grows with the sample size, so with about 80,000 training encounters even a tiny association will give a very small p value. Because of this, we also report Cramér's V, which rescales the statistic into the range 0 to 1:

$$V = \sqrt{\frac{\chi^2}{n \cdot (\min(r,k) - 1)}} \tag{2.3}$$

where n is the number of observations and r and k are the numbers of rows and columns of the contingency table. A value close to 0 means a very weak association and a value close to 1 means a very strong one. In this way, V tells us how strong a relationship is, while the p value only tells us whether it is likely to be real. We use V to rank categorical features by their relationship with the target.

Bootstrap confidence intervals. Let the test set be resampled B times, each time producing a bootstrap estimate of a metric M. The 95% percentile interval is $[Q(0.025), Q(0.975)]$ of the B estimates. Standard bootstrap resamples encounters independently, which assumes independent observations. Here the same patient can contribute several encounters, so the cluster bootstrap resamples patients with replacement and keeps all encounters of each drawn patient. This preserves within-patient correlation and avoids intervals that are too narrow. We use $B = 2,000$ replicates; all $2,000$ were valid in every case.

Paired bootstrap comparison and Holm correction. To compare a reference

model A with another model B, the same bootstrap sample of patients is scored by both models, and the difference $\Delta = M(A) - M(B)$ is recorded in each replicate. The 95% interval of Δ and a two-sided bootstrap p-value are computed as:

$$p = 2 \cdot \min\left\{ \frac{\#(\Delta \le 0) + 1}{B+1}, \frac{\#(\Delta \ge 0) + 1}{B+1} \right\}$$

When several models are compared with the same reference, the family-wise error is controlled with the Holm step-down procedure. The p-values are sorted ascending $p_{(1)} \le \cdots \le p_{(m)}$, and the adjusted value is:

$$\tilde{p}_{(i)} = \max_{j \le i} \min\{1, (m - j + 1) \cdot p_{(j)}\}$$

With $B = 2{,}000$ the smallest attainable raw p-value is about 0.0005, so with $m = 8$ comparisons the smallest adjusted value is 0.008; this value therefore means "below the resolution of the test" rather than an exact probability.

DeLong and McNemar tests. The DeLong test compares two correlated ROC-AUCs using the theory of U-statistics, and McNemar's test compares the paired error patterns of two classifiers at a fixed threshold using the counts of discordant pairs, with statistic $(b-c)^2/(b+c)$. Both assume independent observations, which is violated when the same patient contributes several encounters. For this reason the experiments in this report rely on the patient-cluster paired bootstrap, which can be applied to any metric (AP, F1, recall, specificity) and respects the clustering; DeLong and McNemar are discussed here as the classical alternatives.

Calibration. A model is calibrated if, among encounters given predicted probability p, the observed fraction of positives is close to p. We assess this with a reliability curve (10 quantile bins, mean predicted probability against observed positive fraction), the Brier score (BS), and the log loss (LL):

$$\mathrm{BS} = \frac{1}{N} \sum_{i=1}^{N} (p_i - y_i)^2$$

$$\mathrm{LL} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)]$$

For reference, a constant prediction equal to the prevalence π gives $\mathrm{BS} = \pi(1-\pi)$ and LL equal to the binary entropy of π. The expected calibration error (ECE) summarises the reliability curve as:

$$\mathrm{ECE} = \sum_{b} \frac{n_b}{N} \left| \mathrm{observed}_b - \mathrm{predicted}_b \right|$$

Decision-curve analysis. For a threshold probability p_t, the net benefit (NB) of a model is:

$$\mathrm{NB} = \frac{\mathrm{TP}}{N} - \frac{\mathrm{FP}}{N} \cdot \frac{p_t}{1 - p_t}$$

which expresses true positives in units of false positives weighted by the odds of the threshold (Vickers and Elkin, 2006). It is compared with "treat everyone":

$$\mathrm{NB} = \pi - (1 - \pi) \cdot \frac{p_t}{1 - p_t}$$

and "treat no one" (NB $= 0$). A model is useful at a threshold if its net benefit exceeds both references.

2.4 Chapter 2 conclusion

This chapter placed our models in the usual range of methods for readmission prediction. Logistic regression, a shallow decision tree and a random forest serve as reference points, and gradient-boosted trees, in the form of CatBoost, are the candidate for the final model. Boosting fits the data sequentially by following the gradient of the Logloss, and the rare positive class can be handled either by reweighting the loss or by moving the decision threshold. Which of these choices actually helps on this dataset is an empirical question, and it is answered with grouped cross-validation in Chapter 3 and Chapter 4.

CHAPTER 3. RESEARCH METHODOLOGY

3.1 Overall architecture of the pipeline

3.1.1 Design objectives

The pipeline was designed around four requirements. The first is that nothing from the validation or test data may influence training. Scalers and encoders are fitted inside scikit-learn pipelines, and they are refitted on the training part of each cross-validation fold. The second is that patients never cross a split boundary, since one patient may have several encounters. The third is that every model is compared on equal terms, with the same preprocessing, the same folds and the same metric. The fourth is that the test set is used only once, after all choices (model, hyperparameters, threshold) have been made.

3.1.2 Four functional blocks (C–F)

Block C (feature engineering and preprocessing) takes the cleaned tables, adds seven engineered features and applies robust scaling and one-hot encoding. Block D (candidate models and class-imbalance handling) trains the eight candidates and ranks them by grouped cross-validated AP and by AP on the validation split. Block E (final classifier) tunes the best candidate, CatBoost, with a randomised search. Block F (probability output and decision threshold) converts the predicted probabilities into class labels using a threshold chosen on the validation split, and the fixed rule is then applied to the test set.

3.1.3 Pipeline diagram

C. Feature engineering and preprocessing $\rightarrow$ D. Candidate models (8) and grouped CV $\rightarrow$ E. CatBoost tuning (randomised search) $\rightarrow$ F. Probability and threshold selection

Figure 3.1 Overall structure of the pipeline

The figure shows the flow of the work. The cleaned training data go through Block C, the processed features are used in Block D to rank the candidates, the winner is tuned in Block E, and Block F fixes the decision rule. The test set enters only after Block F is complete.

3.2 Data preprocessing and feature engineering

3.2.1 Data cleaning, ICD-9 grouping and feature encoding

The preprocessing strategy distinguishes predefined, rule-based transformations from decisions learned from the data. Data-dependent decisions, including near-constant feature removal, category selection, and percentile-based outlier limits, are fitted on the

training set only and then applied unchanged to the test set.

Excluding encounters with death or hospice discharge. Encounters with discharge disposition codes associated with death or hospice care (IDs 11, 13, 14, 19, 20, and 21 in IDS_mapping.csv) are excluded because these outcomes make subsequent readmission unsuitable as a comparable prediction outcome. This removes 2,423 encounters (approximately 2.4% of the original dataset), leaving 99,343 encounters for analysis.

Merging unknown administrative codes. Several codes in the administrative ID columns represent unknown or unavailable information. These are consolidated into a single label, Unknown: discharge disposition (IDs 18, 25, 26), admission source (IDs 9, 15, 17, 20, 21), and admission type (IDs 5, 6, 8). This reduces unnecessary fragmentation of categories that carry similar meanings.

Handling missing values. The placeholder ? is first converted to a standard missing-value representation. The weight column is then removed because of its extensive missingness. Missing values in race, payer_code, and medical_specialty are assigned the label Unknown, while missing values in max_glu_serum and A1Cresult are labeled Not tested. Missing diagnosis codes are handled separately during ICD-9 grouping.

Grouping diagnoses by ICD-9. The three raw diagnosis columns contain hundreds of distinct codes, making direct use of the original codes less practical for modeling. A rule-based mapping function assigns codes beginning with V and E to Supplementary_V and External_cause_E, respectively. Numeric codes are mapped into broader ICD-9-based groups, including Diabetes, Circulatory, Respiratory, Digestive, Genitourinary, Musculoskeletal, Injury, Neoplasms, Infectious, Endocrine_other, Blood, Mental, Nervous_SenseOrgans, Pregnancy_Childbirth, Skin, Congenital, Perinatal, and Symptoms_signs. Codes in the diabetes range are separated from other endocrine conditions, while selected symptom codes are assigned to relevant organ-system groups. Missing codes receive the label Missing, and codes that cannot be matched receive Other. The resulting features, diag_1_group, diag_2_group, and diag_3_group, preserve broad diagnostic information while reducing the number of distinct levels. The original diagnosis columns are dropped before the processed files are saved.

Removing near-constant medication columns. Of the 23 original medication columns, 15 are removed because their most frequent value accounts for at least 99% of the training encounters. This decision is made using the training set only, and the same columns are removed from the test set. The removed columns are examide, citoglipton, acetohexamide, troglitazone, tolbutamide, miglitol, tolazamide, chlorpropamide, acarbose, nateglinide, glimepiride-pioglitazone, metformin-pioglitazone, metformin-

rosiglitazone, glipizide-metformin, and glyburide-metformin. The five combination-drug columns are included in this list. Eight medications remain: metformin, repaglinide, glimepiride, glipizide, glyburide, pioglitazone, rosiglitazone, and insulin. Insulin is the most variable medication, with its most frequent value, No, accounting for approximately 46.6% of training encounters.

Categorical preparation. The remaining categorical variables consist of 15 base columns and the eight retained medication columns. These variables are converted to the categorical data type, with the category vocabulary determined from the training set only. Test-set values that were not observed during training are assigned the special label Other_unseen, ensuring that the test data do not introduce additional categories into the learned vocabulary.

Outlier treatment. Five count variables with right-skewed distributions are winsorized: num_medications, num_lab_procedures, number_inpatient, number_emergency, and number_outpatient. The lower and upper limits are determined from the 1st and 99th percentiles of the training set, respectively, using nearest-value interpolation. Values outside these limits are capped at the corresponding boundary. The same limits are applied to the test set, preventing test data from influencing the thresholds. This procedure retains all encounters while limiting the influence of extreme count values on subsequent modeling.

Table 3.1 Winsorization limits and capped values

Variable	Lower limit	Upper limit	Train: below	Train: above	Test: below	Test: above
`num_medications`	3	43	584	757	133	171
`num_lab_procedures`	1	84	0	741	0	208
`number_inpatient`	0	6	0	534	0	170
`number_emergency`	0	3	0	726	0	204
`number_outpatient`	0	5	0	721	0	182

Winsorization is preferred over deleting extreme encounters because large counts are likely to reflect genuine high healthcare utilization and the positive class is rare. The effect of the treatment is illustrated in Figure 4.9. The main trade-off is a loss of resolution among very high values; for example, all values of `number_inpatient` above 6 (about 0.7% of training encounters) are recorded as 6.

After cleaning, the model input contains 15 numerical and 23 categorical columns. The numerical columns are scaled with a RobustScaler, which centres each feature on its median and divides by its interquartile range. We chose it over standardisation because the count variables (number of medications, lab procedures, prior visits) have long right tails, and a few extreme values would otherwise dominate the mean

and the standard deviation. The categorical columns are one-hot encoded with handle_unknown="ignore", so a category that appears only in a validation or test fold produces an all-zero block and does not raise an error. The three identifier columns admission_type_id, discharge_disposition_id and admission_source_id are stored as numbers but are codes, not quantities, so they are treated as categorical. Both transformations are wrapped in one ColumnTransformer that sits inside every model pipeline. As a result, the median, the quantiles and the list of categories are always learned from the training rows of the current fit only.

3.2.2 Labeling and missing-data handling

Labeling. We create the binary column target with target = 1 if readmitted equals "<30" and 0 otherwise. The original three class column is then dropped, so that it cannot leak into the features.

Missing values. We treat each type of missingness according to its meaning.

First, the "?" entries are converted to proper missing values. The column weight is missing in 96.9% of encounters, so it contains almost no usable information, and we drop it entirely.

Second, for medical_specialty (49.1% missing), payer_code (39.6%), and race (2.2%), we ask whether the fact that a value is missing is itself related to readmission. We run a chi square test between a missing indicator and the target on the training set. The results are:

Table 3.2 Chi square test of missingness vs target (training set)

Column	Missing rate	p value	Conclusion
`medical_specialty`	48.92%	1.98e-05	Associated with target
`payer_code`	39.68%	0.0738	No clear association
`race`	2.28%	1.98e-05	Associated with target

For medical_specialty and race, missingness is related to readmission, so simply deleting or imputing these values with the most common category could hide a real signal. For all three columns we therefore keep the rows and replace missing values with an explicit label, "Unknown". This lets the model learn from the missingness itself. This is a fixed rule, not a statistic learned from the data, so it is applied identically to the training and test sets.

Third, for max_glu_serum and A1Cresult, the 94.7% and 83.3% missing rates come from the string "None" in the original file, which pandas reads as missing. It

means the test was not performed, which is not the same as unknown. We therefore fill them with a separate label, "Not tested".

After these steps, only the three diagnosis columns still contain a small number of missing values in the training set (15 in diag_1, 296 in diag_2, and 1,169 in diag_3). These are handled by the ICD 9 grouping step, which assigns the label "Missing" to them, so the final modelling data has no missing entries.

After cleaning, the model input contains 15 numerical and 23 categorical columns. The numerical columns are scaled with a `RobustScaler`, which centres each feature on its median and divides by its interquartile range. We chose it over standardisation because the count variables (number of medications, lab procedures, prior visits) have long right tails, and a few extreme values would otherwise dominate the mean and the standard deviation. The categorical columns are one-hot encoded with `handle_unknown="ignore"`, so a category that appears only in a validation or test fold produces an all-zero block and does not raise an error. The three identifier columns `admission_type_id`,`discharge_disposition_id` and `admission_source_id` are stored as numbers but are codes, not quantities, so they are treated as categorical. Both transformations are wrapped in one `ColumnTransformer` that sits inside every model pipeline. As a result, the median, the quantiles and the list of categories are always learned from the training rows of the current fit only.

3.2.3 Advanced feature engineering

The cleaned data already include the counts of outpatient, emergency and inpatient visits in the year before the encounter, together with the number of lab procedures, procedures and medications. We derive seven additional features from these columns. They are row-wise calculations that use neither the target nor any statistic from other rows, so they cannot leak information between splits, and the same function is applied to the training and test tables.

The lower clip of 1 on `time_in_hospital` prevents division by zero. The two indicator variables give tree-based models a ready-made split on "any prior stay" and let logistic regression use it as a simple on/off effect. After this step the training table has 38 predictors (the original 31 plus the seven new ones) and the target.

3.2.4 Patient-level data splitting

As explained in Section 1.2.2, an encounter level random split would put different encounters of the same patient in both sets and lead to optimistic results. We avoid this by splitting by patient with scikit learn's GroupShuffleSplit, using patient_nbr as the group, test_size = 0.2, and random_state = 42. All encounters of a given patient go entirely to either the training set or the test set.

Table 3.3 Engineered features.

Feature	Definition	Reason for including it
total_previous_visits	number_outpatient + number_emergency + number_inpatient	Overall contact with the health system in the previous year
has_prior_inpatient	1 if number_inpatient > 0, else 0	Prior admissions are the strongest known risk factor for readmission
has_prior_emergency	1 if number_emergency > 0, else 0	Marks patients who have used emergency care
medications_per_day	num_medications / max(time_in_hospital, 1)	Medication load relative to the length of stay
labs_per_day	num_lab_procedures / max(time_in_hospital, 1)	Intensity of laboratory work-up
procedures_per_day	num_procedures / max(time_in_hospital, 1)	Intensity of procedures
treatment_intensity	num_lab_procedures + num_procedures + num_medications	Rough single measure of how much care the stay required

The result is 79,541 training encounters (about 80%) and 19,802 test encounters (about 20%). Two checks confirm that the split behaves as intended. The number of patients appearing in both sets is zero, and the positive rate is similar in the two sets (11.44% in training and 11.19% in test). Note that the split is grouped but not stratified. We did not need stratification because the positive rates already match closely.

To preserve patient-level information throughout the pipeline, we save the patient_nbr associated with each row in train_patient_groups.csv and test_patient_groups.csv, keeping the identifiers aligned with the corresponding feature tables. The columns patient_nbr and encounter_id are then removed from the feature tables so that patient and encounter identifiers cannot be used directly as model predictors.

However, patient identifiers are retained separately as grouping metadata because multiple encounters may belong to the same patient. This information supports patient-level separation during validation and cross-validation, as described in the modelling section, and is also used for cluster bootstrap analysis during evaluation. Rather than resampling individual encounters independently, the bootstrap procedure resamples patients as clusters to account for the dependence between repeated encounters belonging to the same patient. Retaining these group identifiers therefore allows the evaluation to

reflect the dataset's patient-level structure while keeping identifiers out of the model's input features.

Inside the training set we hold out a validation split with GroupShuffleSplit (20% of the patients, random_state = 42), using patient_nbr as the group. This gives a fit set of 63,807 encounters and a validation set of 15,734 encounters. An explicit check confirms that the two sets share no patients, and the positive rate is almost the same in both (0.1143 in the fit set and 0.1145 in the validation set). The fit set is used for model comparison and tuning, and the validation set is used to compare the candidates and to choose the decision threshold.

Cross-validation inside the fit set uses StratifiedGroupKFold with shuffling and random_state = 42. This splitter keeps all encounters of a patient in the same fold and tries to keep the share of positives similar across folds. We use 3 folds for the comparison of the eight candidates, because several of the models are slow to train, and 5 folds for the hyperparameter search, where a more stable estimate of AP is worth the extra cost.

Inside the training set we hold out a validation split with `GroupShuffleSplit` (20% of the patients, `random_state = 42`), using `patient_nbr` as the group. This gives a fit set of 63,807 encounters and a validation set of 15,734 encounters. An explicit check confirms that the two sets share no patients, and the positive rate is almost the same in both (0.1143 in the fit set and 0.1145 in the validation set). The fit set is used for model comparison and tuning, and the validation set is used to compare the candidates and to choose the decision threshold.

Cross-validation inside the fit set uses `StratifiedGroupKFold` with shuffling and `random_state = 42`. This splitter keeps all encounters of a patient in the same fold and tries to keep the share of positives similar across folds. We use 3 folds for the comparison of the eight candidates, because several of the models are slow to train, and 5 folds for the hyperparameter search, where a more stable estimate of AP is worth the extra cost.

3.3 Candidate models and class-imbalance handling

We compare four model families, each trained in two versions: one with the default loss and one with class weights. All eight candidates share the same preprocessing block and the same random seed (42), so differences in performance come from the model and not from the data preparation. The settings are fixed by hand and are not tuned at this stage, because the purpose of this step is to choose a family and not to find the best configuration of each one.

Logistic regression is the linear reference. The decision tree is depth-limited to keep it interpretable and to avoid fitting noise. The random forest represents bagging and

Table 3.4 Candidate models.

Model	Main settings
Logistic Regression	`max_iter = 1000`
Decision Tree	`max_depth = 8`, `min_samples_leaf = 20`
Random Forest	`n_estimators = 200`, `max_depth = 12`, `min_samples_leaf = 10`, `n_jobs = -1`
CatBoost	`iterations = 200`, `depth = 6`, `learning_rate = 0.1`, `loss = Logloss`
+ Class Weight versions	`class_weight = "balanced"` (LR, DT, RF); `auto_class_weights = "Balanced"` (CatBoost); other settings unchanged

CatBoost represents boosting. The class-weight versions let us measure, for each family, whether reweighting the loss helps ranking performance on this dataset. The primary criterion is AP, and the random baseline is the positive rate of the fit set (0.1143), so a model with AP of 0.23 is about twice as good as random guessing.

3.4 Final classifier: CatBoost

CatBoost was chosen as the final classifier because it had the highest cross-validated AP and the highest validation AP among the eight candidates (Section 4.3.1). It also has a relatively small set of influential hyperparameters, which makes a random search of modest size reasonable.

The final pipeline consists of the preprocessing block described in Section 3.2.1 followed by a `CatBoostClassifier` with the `Logloss` objective, `random_seed = 42` and the hyperparameters returned by the search (Section 3.6). No class weights are used, because the comparison showed that the unweighted version ranked patients better, and imbalance is handled by the threshold in Section 3.5. The pipeline is created by a single function, so that exactly the same configuration is used for the validation fit and for the final fit.

One design limitation should be stated. Because the categorical columns are one-hot encoded before they reach CatBoost, the model does not use its native treatment of categorical features. This was done so that all candidates see identical input and the comparison is fair. A configuration that passes the categorical columns to CatBoost directly might perform differently, and we list it as a possible extension in the conclusion.

3.5 Probability output and decision threshold

The model outputs the estimated probability of the positive class (the column of `predict_proba` that corresponds to class 1). Turning this probability into a label requires a threshold. With only 11% positives, a threshold of 0.5 would label very few patients as high-risk and give a low recall, so we select the threshold from the data.

The selection is done on the validation split. A CatBoost pipeline with the tuned hyperparameters is fitted on the fit set only and used to predict the validation patients. We compute the precision-recall curve for these predictions, calculate F1 at every candidate threshold, and keep the threshold with the highest F1. This gave a threshold of 0.145. The value is then fixed. The final model is refitted on the complete training set and applied to the test set with this same cut-off, and the test labels play no role in the choice.

Maximising F1 treats false positives and false negatives as equally costly, which is a simplifying assumption. In a hospital the two errors do not cost the same, so Section 4.5 looks at other operating points and at a cost-sensitive choice of threshold. A small technical point is that the threshold is selected on a model fitted to 80% of the training data and then used with a model fitted to all of it. The two models should produce similar probability scales, but they are not identical, and we note this as a minor limitation.

3.6 Training strategy and hyperparameter optimization

Training follows four steps. First, the eight candidates are ranked with 3-fold grouped cross-validation on the fit set and also scored once on the validation split. Second, CatBoost is tuned on the fit set. Third, the threshold is chosen on the validation split. Fourth, the final pipeline is refitted on all training rows and evaluated once on the test set.

The tuning uses `RandomizedSearchCV` with 20 random configurations and 5 grouped folds (`StratifiedGroupKFold`), which makes 100 model fits, and AP as the scoring function. Random search was preferred to a grid because it covers a wide range of values for each parameter with the same number of fits, and it is usually just as effective when only a few parameters matter. The search ranges are given in Table 3.5.

The best configuration had depth $= 5$, iterations $= 764$, l2_leaf_reg $= 4.91$ and learning_rate $= 0.0582$, with a mean cross-validated AP of 0.2336. Compared with the default settings (depth 6, 200 iterations, learning rate 0.1), the search chose a lower learning rate and about twice as many iterations, which is the usual pattern when a boosting model is set to learn more slowly. The search used only the fit set, so the validation split remained untouched until the threshold step.

Table 3.5 Hyperparameter search space for CatBoost.

Hyperparameter	Distribution	Range sampled
depth	`randint(4, 10)`	integers 4 to 9
learning_rate	`uniform(0.01, 0.2)`	0.01 to 0.21
l2_leaf_reg	`uniform(1, 10)`	1 to 11
iterations	`randint(200, 800)`	integers 200 to 799

3.7 Chapter 3 conclusion

This chapter described a pipeline in which each stage has a defined role and in which the splits are made by patient from start to finish. Seven engineered features were added to the cleaned data, and robust scaling and one-hot encoding were placed inside the model pipelines. Eight candidates were set up to compare four model families with and without class weights, and CatBoost was chosen as the final classifier. Its hyperparameters are tuned by a 20-configuration random search with grouped 5-fold cross-validation, and the decision threshold is chosen by maximising F_1 on a patient-disjoint validation split. The results of these steps are presented in Chapter 4.

CHAPTER 4. EXPERIMENTS AND EVALUATION

4.1 Experimental objectives

The experiments are organised around three axes, because a model that ranks patients slightly better than chance is only useful if it can be shown to be better than reasonable alternatives, usable under real constraints and trustworthy across patients.

1. **Performance relative to baselines.** Does the final model rank readmission risk better than trivial policies (flag nobody, flag everybody) and than the other seven candidate models, and are the differences larger than sampling noise? This is answered with AP and ROC-AUC, patient-cluster confidence intervals and Holm-corrected paired tests (Sections 4.3.2–4.3.4).

2. **Operational value.** At a realistic operating point, how many patients are flagged, how many true readmissions are found, and under what cost assumptions is the model better than not using it? This is answered with threshold analysis, cost-sensitive analysis, net benefit and lift (Sections 4.4.3 and 4.5).

3. **Reliability.** Are the predicted probabilities meaningful, where does the model fail, and does performance differ across age, gender, race, diagnosis and discharge groups? This is answered with calibration, error and subgroup analysis (Section 4.6).

Generalisation to an external dataset is deliberately excluded from the objectives, because no second dataset with the same variables is available.

4.2 Experimental setup

4.2.1 Computing environment

All experiments were run in Python 3.12.10 inside a virtual environment, using Jupyter notebooks. The main libraries are `pandas` and `NumPy` for data handling, `scikit-learn` for preprocessing, splitting, baseline models and metrics, `SciPy` for the parameter distributions of the random search, `CatBoost` for the gradient-boosting models, and `joblib` for saving the fitted pipeline. All models were trained on the CPU, and cross-validation was run in parallel (`n_jobs` $= -1$). A random seed of 42 was used in every splitter and model, which makes the results reproducible on the same library versions.

The notebook writes its results to an `outputs` folder: `validation_results.csv`, `cv_results.csv`, `best_params.json`, `test_predictions.csv`, `test_metrics.csv` and the fitted pipeline `final_model.joblib`. The evalua-

tion notebook reads these files, so the figures in this chapter can be regenerated without retraining.

4.2.2 Training strategy

The training strategy follows Section 3.6. The 79,541 training encounters were split by patient into a fit set (63,807) and a validation set (15,734). Models were compared with 3-fold `StratifiedGroupKFold` cross-validation on the fit set and were also scored once on the validation set. CatBoost was tuned with a 20-configuration random search over 5 grouped folds. The decision threshold was chosen on the validation set. The final model was then refitted on the full training set, and the test set (19,802 encounters) was used once for the final results.

4.2.3 Dataset

The experiments use the Diabetes 130 US Hospitals dataset, which covers inpatient encounters of patients with diabetes from 130 hospitals between 1999 and 2008. The raw file diabetic_data.csv contains 101,766 encounters and 50 columns, and it is accompanied by IDS_mapping.csv, which explains the administrative ID codes. Each encounter_id is unique and there are no fully duplicated rows. The encounters belong to 71,518 distinct patients, of whom 16,773 have more than one encounter.

After removing patients who died or went to hospice, 99,343 encounters remain. These are divided by patient into 79,541 training encounters and 19,802 test encounters. The final tables have the binary target target plus the cleaned features (demographics, administrative IDs, counts, laboratory results, eight medications, the change and diabetesMed indicators, and the three diagnosis groups).

Table 4.1 Dataset summary after the split

Item	**Value**
Raw encounters / columns	101,766 / 50
Unique patients	71,518
Patients with more than one encounter	16,773
Encounters removed (expired or hospice)	2,423
Encounters after cleaning	99,343
Training / test encounters	79,541 / 19,802
Positive rate (train / test)	11.44% / 11.19%
Patients appearing in both sets	0

4.2.4 Exploratory data analysis

We now look at the cleaned training set to understand which features are related to early readmission. All figures use the training data only.

Target distribution. The target is strongly imbalanced, with about 11% positives (Section 1.2.1).

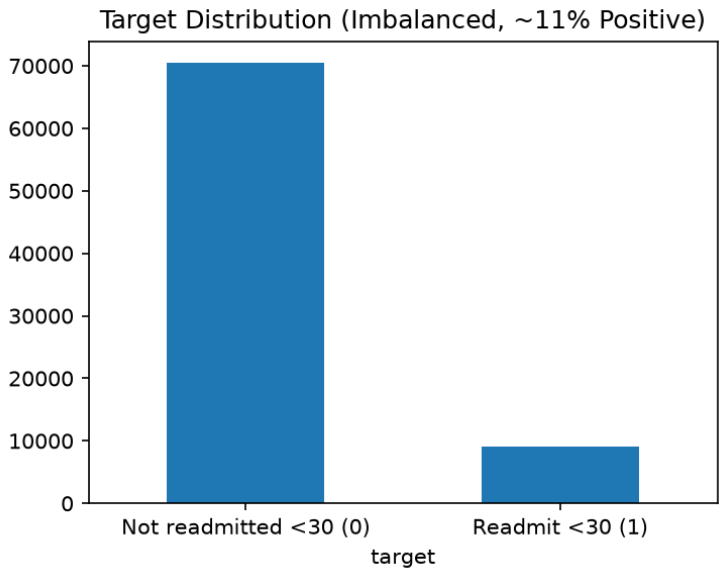


Figure 4.1 Distribution of the binary target in the training set. Only about 11% of encounters are readmitted within 30 days

Age. Most encounters belong to older patients, with the largest groups in the 60 to 80 age range. Because there are so many more older patients, the raw counts of early readmissions are also highest there. We should be careful to compare proportions and not only counts, since the plot of counts mainly reflects how many patients each age group has.

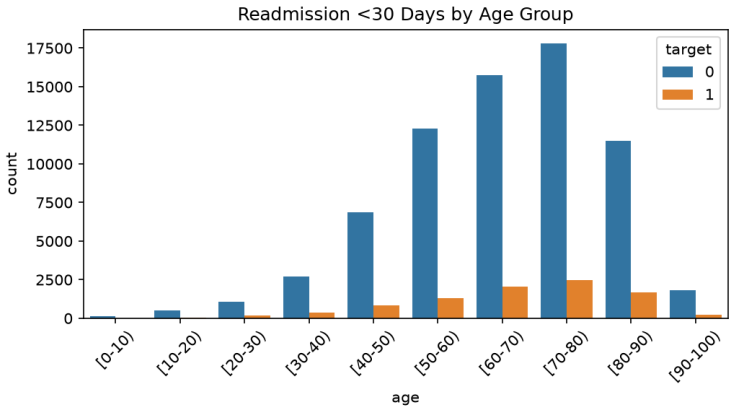


Figure 4.2 Number of encounters by age group, split by readmission within 30 days.

Length of stay. The correlation between time_in_hospital and the readmission target is weak (approximately 0.046), suggesting that length of stay alone has limited

linear association with readmission. However, this does not necessarily mean that the variable has no predictive value, as its relationship with readmission may depend on other clinical factors.

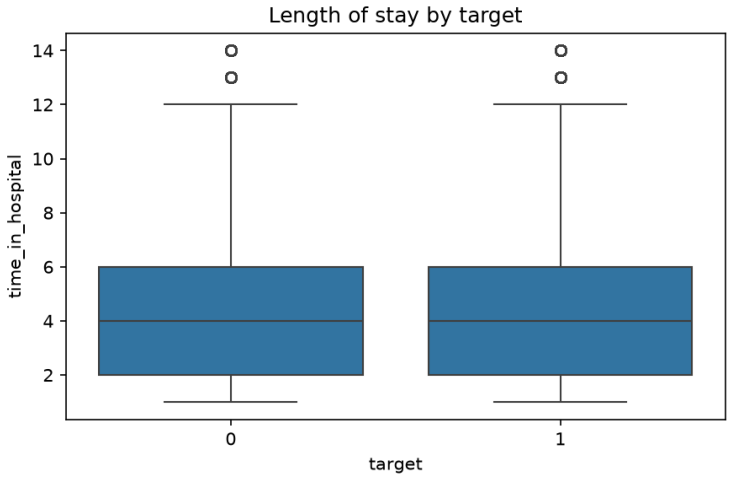


Figure 4.3 Length of Hospital Stay by Readmission Outcome

Primary diagnosis. The distribution of primary diagnosis groups is highly imbalanced. Circulatory diseases account for the largest number of observations, followed by respiratory diseases, digestive diseases, and diabetes. Readmission cases (target = 1) appear across multiple diagnosis groups, although the number of cases varies considerably between groups.

Because the figure displays raw counts rather than within-group readmission rates, these differences should not be interpreted directly as differences in readmission risk. Further analysis using group-specific rates or an appropriate association measure, such as Cramér's V, is needed to assess the relationship between primary diagnosis and readmission.

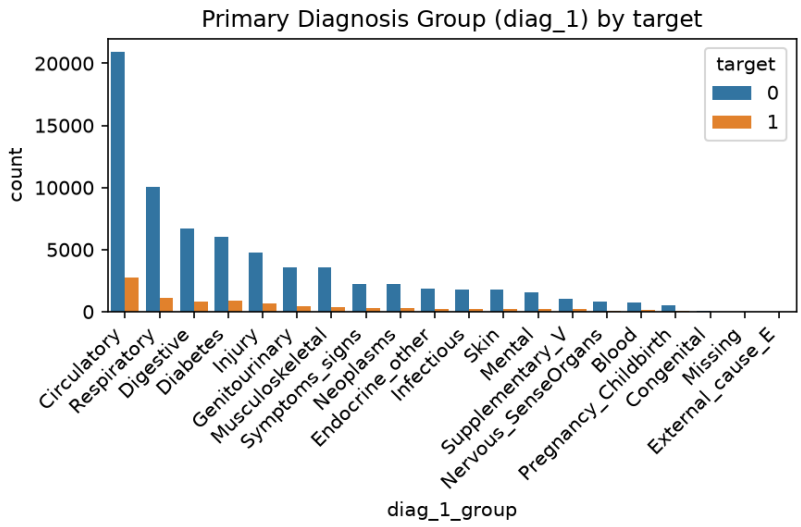


Figure 4.4 Distribution of Readmission Outcomes Across Primary Diagnosis Groups

Numerical variables and the target. The correlation analysis shows that all eight numerical clinical variables have relatively weak correlations with the readmission target. number_inpatient has the strongest positive correlation (0.167), followed by number_emergency (0.073), number_diagnoses (0.054), time_in_hospital (0.046), num_medications (0.041), number_outpatient (0.027), and num_lab_procedures (0.025). In contrast, num_procedures has a small negative correlation (approximately 0.015).

Among these variables, previous inpatient visits show the strongest linear association with readmission. This suggests that a patient's history of hospitalization may be more informative than individual measures of treatment intensity or service utilization. Nevertheless, the low correlation values indicate that no single numerical variable captures the readmission outcome particularly well on its own.

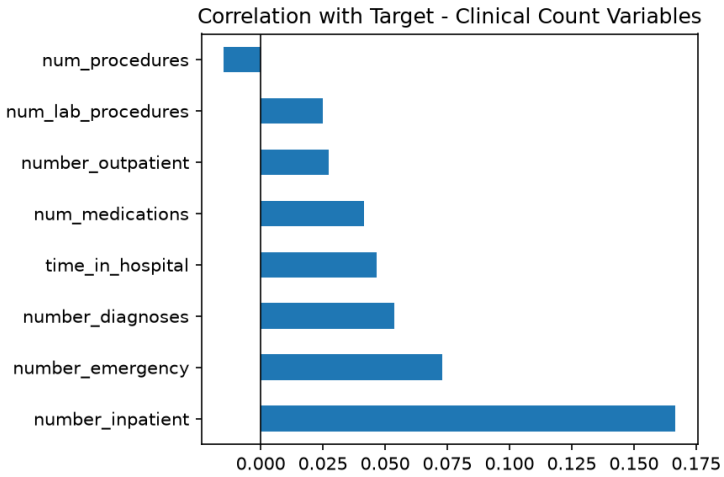


Figure 4.5 Correlation Between Numerical Clinical Variables and the Readmission Target

A more informative pattern emerges when the number of previous inpatient visits is grouped into four categories: 0, 1, 2, and 3 or more visits. The readmission rate increases consistently across these categories, rising from approximately 8.6% among patients with no previous inpatient visits to 13.3% for one visit, 18.3% for two visits, and 26.2% for three or more visits.

This pattern suggests a clear positive association between previous inpatient utilization and readmission. Patients with a more extensive hospitalization history may represent a group requiring closer follow-up and more comprehensive discharge planning. However, this descriptive relationship does not establish causality, and other clinical and demographic factors may also contribute to readmission risk.

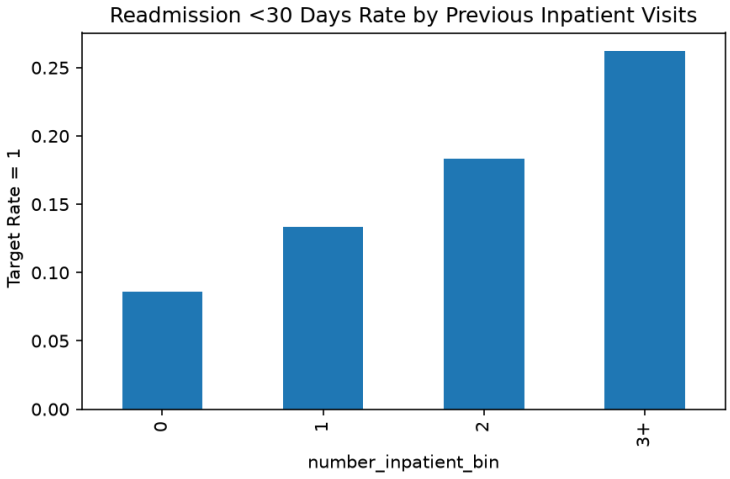


Figure 4.6 Readmission Rate by Number of Previous Inpatient Visits

Correlation among numerical variables. The correlation matrix reveals several positive relationships among the numerical clinical variables. The strongest is between num_medications and time_in_hospital (0.47), followed by num_medications and num_procedures (0.38), and num_lab_procedures and time_in_hospital (0.32). Previous inpatient and emergency visits also show a positive correlation (0.28).

These relationships suggest that patients with longer hospital stays may receive more medications and undergo more laboratory tests, while patients with a history of inpatient visits may also have more emergency visits. However, correlation alone does not establish the underlying clinical mechanisms.

The observed correlations are mostly moderate or weak, with no pair exceeding 0.50. Therefore, severe multicollinearity is not immediately evident from this matrix, although further assessment may be necessary when fitting linear models.

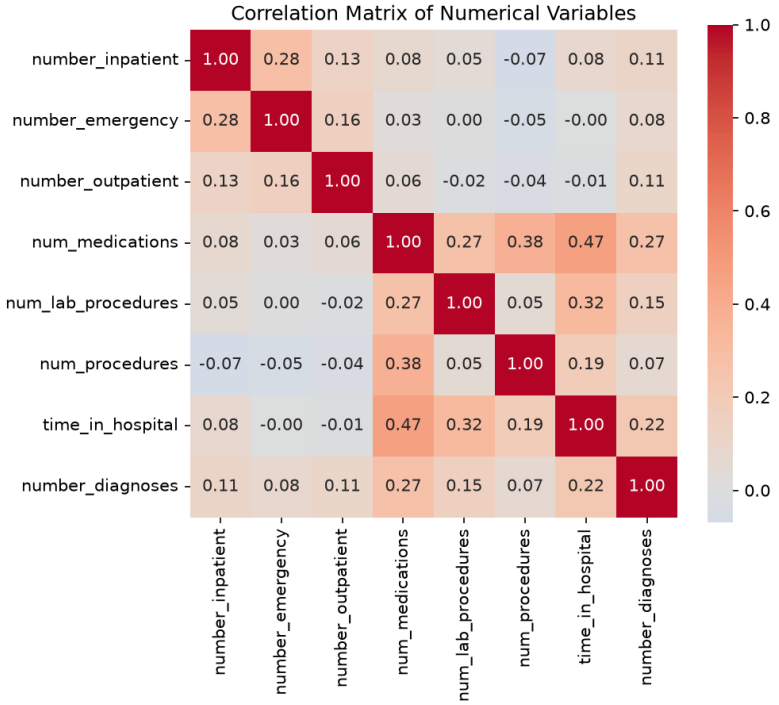


Figure 4.7 Correlation Matrix of Numerical Clinical Variables

Medications. The medication analysis examines insulin, metformin, glipizide, glyburide, pioglitazone, rosiglitazone, change, and diabetesMed in relation to the readmission target.

The distributions are highly imbalanced across medication categories. For most individual drugs, the majority of observations fall into the No category, while insulin and metformin have larger numbers of patients in their respective treatment categories. The summary variable diabetesMed also shows that medication use is much more common than non-use in the dataset.

However, because the plots display counts rather than readmission rates within each category, they do not establish which medication is associated with a higher risk of readmission. In addition, medication use may reflect disease severity, treatment history, or other clinical factors rather than an independent effect of the medication itself. Comparing normalized readmission rates or using an appropriate categorical association measure would provide a more reliable assessment.

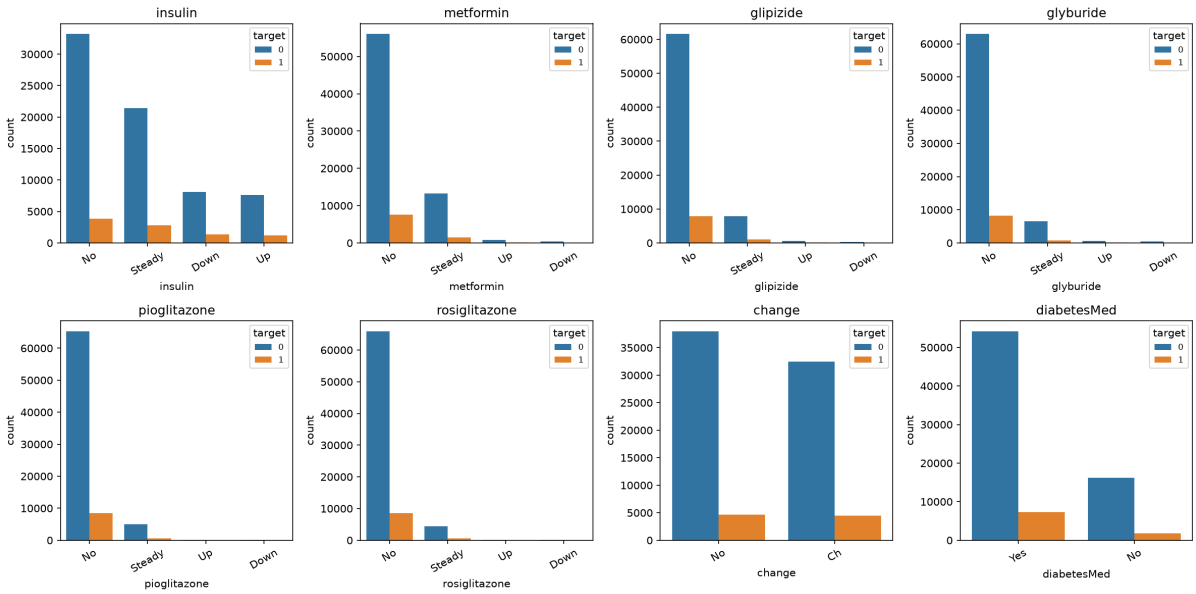


Figure 4.8 Distribution of Readmission Outcomes Across Medication-Related Variables

Outliers. The five clinical count variables are strongly right-skewed: most encounters have small values, while a small number have very large ones (Figure 4.9, top row). Such values most likely reflect genuine differences in healthcare utilization, for example patients with many prior admissions, rather than data-entry errors. Removing these encounters would therefore discard clinically meaningful information, including some of the already scarce positive cases. We instead applied winsorization at the 1st and 99th percentiles of the training set, which modifies fewer than 2% of encounters for each variable. Figure 4.9 compares the distributions before (top) and after (bottom) the treatment. Extreme values are capped at the limits, while the medians and quartiles remain unchanged, so the main body of each distribution is preserved.

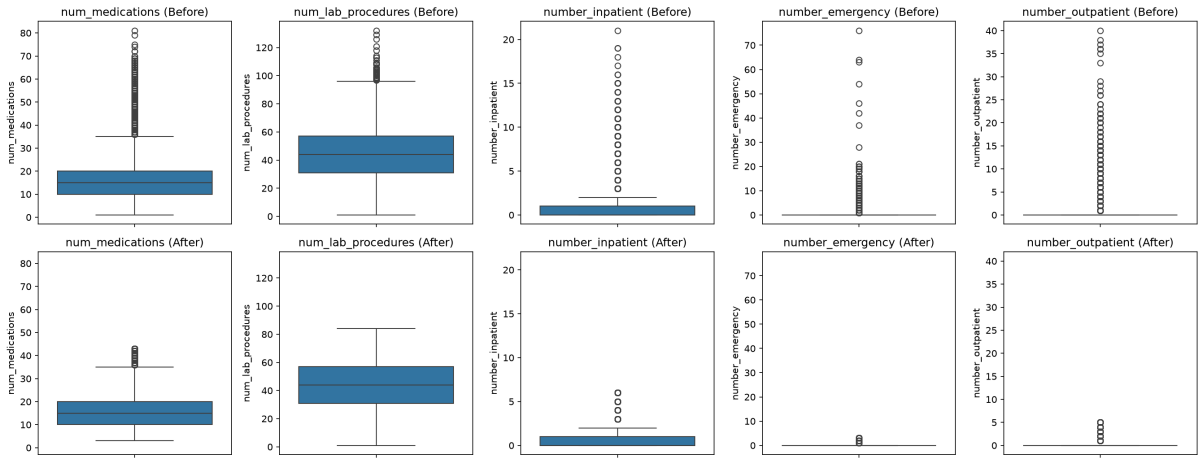


Figure 4.9 Distributions of the five winsorized count variables before (top row) and after (bottom row) winsorization at the training-set 1st and 99th percentiles. Each variable uses the same y-axis scale in both rows.

Association of categorical variables with the target. We finally ranked the categorical variables by Cramér's V against the target, together with the chi square p value. The results are shown below.

Table 4.2 Cramér's V and p-value for categorical features

No.	Column	cramers_v	p_value
4	discharge_disposition_id	0.112332	8.657883e-201
8	diag_3_group	0.044049	3.180541e-23
9	insulin	0.043395	2.928568e-32
6	diag_1_group	0.040145	3.205707e-18
2	age	0.038690	2.074287e-21
7	diag_2_group	0.037305	5.943767e-15
5	admission_source_id	0.024804	9.192013e-06
11	diabetesMed	0.024642	3.662027e-12
12	A1Cresult	0.022112	1.832172e-08
3	admission_type_id	0.018464	5.411959e-05
10	change	0.017483	8.196270e-07
0	race	0.017052	3.190969e-04
13	max_glu_serum	0.014437	8.631246e-04
1	gender	0.004077	5.162407e-01

discharge_disposition_id stands out as the strongest categorical predictor, which is reasonable because where a patient is sent after discharge (home, another facility, and so on) reflects how sick they still are. Diagnosis groups, insulin use, and age form the next tier. All other variables have very weak associations. gender is the only variable with no

significant association (p = 0.516). Note that many p values are tiny simply because the sample is large, which is why we rely on Cramér's V for the strength of the relationship.

No single feature is strongly related to early readmission. The correlations and Cramér's V values are all small, with prior inpatient visits and discharge disposition being the most informative. This suggests that the signal in this dataset is weak and spread across many variables, which is consistent with the difficulty of the task discussed in Chapter 1 and motivates a model that can combine many weak features.

4.2.5 Comparison models

The baselines that we trained ourselves are the eight non-final candidates from Table 3.4: logistic regression, decision tree and random forest, each with and without class weights, and the default CatBoost with class weights. They use the same features, the same preprocessing and the same patient-level splits as the final model, so any difference in score comes from the learning algorithm. The default (untuned) CatBoost is also used as the reference in the tuning ablation. The results of published studies are collected separately in Section 4.3.4.

4.2.6 Evaluation metrics

All metrics are computed from predicted probabilities p and true labels y. Threshold-dependent metrics use the predicted label $\hat{y} = 1$ if $p \geq t$. TP, FP, FN, TN denote the confusion-matrix counts, N is the number of encounters and π is the prevalence.

Ranking metrics (independent of the threshold).

- **ROC-AUC** is the probability that a randomly chosen positive encounter receives a higher score than a randomly chosen negative one. A random ranking gives 0.5. It is secondary here because with a rare positive class it can look acceptable even when precision is low.

- **Average Precision (AP)** is the area under the precision–recall curve, $\mathrm{AP} = \sum_n (R_n - R_{n-1}) \cdot P_n$. A random ranking gives $\mathrm{AP} = \pi = 0.1119$ on the test set. AP is the primary metric because the positive class is rare and AP directly measures how well true readmissions are concentrated at the top of the ranking (Saito and Rehmsmeier, 2015). It is used for model selection, tuning and testing.

Threshold-dependent metrics (computed at the validation-selected threshold).

- **Recall (sensitivity)** $= \frac{\mathrm{TP}}{\mathrm{TP+FN}}$: the share of readmissions that are found.

- **Precision (PPV)** $= \frac{\mathrm{TP}}{\mathrm{TP+FP}}$: the share of flagged patients that are true readmissions.

- **Specificity** $= \frac{\mathrm{TN}}{\mathrm{TN+FP}}$: the share of non-readmitted patients correctly not flagged.

- F_1 **and** F_2: $F_1 = \frac{2PR}{P+R}$ and $F_2 = \frac{5PR}{4P+R}$, the harmonic means of precision and recall, with F_2 weighting recall four times as much as precision in the denominator. F_1 is used to choose the threshold.

- **Matthews correlation coefficient (MCC)** $= \frac{\mathrm{TP}\cdot\mathrm{TN}-\mathrm{FP}\cdot\mathrm{FN}}{\sqrt{(\mathrm{TP+FP})(\mathrm{TP+FN})(\mathrm{TN+FP})(\mathrm{TN+FN})}}$: a balanced correlation between prediction and truth that uses all four cells of the confusion matrix and is robust to class imbalance. It ranges from -1 to 1, with 0 for no better than chance.

- **Flagged %** $= \frac{\mathrm{TP+FP}}{N}$: the share of encounters that would receive an intervention, an operational quantity.

Probability-quality metrics. Brier score, log loss and the reliability curve described in Section 2.3.3, plus ECE.

Why accuracy is misleading. On the test set the share of negatives is $1-\pi =$ 88.81%. A policy that flags nobody therefore achieves an accuracy of 0.8881, higher than the 0.7622 of the final model, although it finds no readmissions (recall 0, F_1 0). Conversely, flagging everybody has accuracy 0.1119 but recall 1. Accuracy rewards ignoring the minority class and is therefore reported only for completeness; AP, recall, precision, F_1 and MCC are the metrics used for judgement.

Uncertainty reporting. Point estimates are accompanied by 95% patient-cluster bootstrap intervals (2,000 replicates), and model comparisons use paired cluster bootstrap with Holm correction (Section 2.3.3).

Table 4.3 Performance comparison between naive reference policies and the tuned CatBoost model on the test set.

Policy (test set)	Accuracy	Recall	Precision	F_1	AP	ROC-AUC
Flag nobody	0.888	0.000	undefined	0.000	0.112	0.500
Flag everybody	0.112	1.000	0.112	0.201	0.112	0.500
Tuned CatBoost (threshold 0.145)	0.762	0.431	0.217	0.288	0.244	0.677

4.3 Model evaluation results

4.3.1 Model selection results (CV and validation)

Table 4.4 shows the 3-fold grouped cross-validation results on the fit set. The random baseline AP is 0.1143, the positive rate of the fit set.

CatBoost without class weights has the best mean AP (0.2270), about twice the

Table 4.4 Grouped 3-fold cross-validation results (fit set).

Model	Mean CV AP	Std CV AP	AP / random baseline
CatBoost	0.2270	0.0052	1.99
Random Forest	0.2192	0.0036	1.92
CatBoost + Class Weight	0.2161	0.0022	1.89
Random Forest + Class Weight	0.2109	0.0008	1.84
Logistic Regression	0.2091	0.0031	1.83
Logistic Regression + Class Weight	0.2073	0.0029	1.81
Decision Tree	0.1920	0.0012	1.68
Decision Tree + Class Weight	0.1899	0.0020	1.66

random baseline. The random forest is second, and logistic regression is close behind it. The two decision trees are clearly the weakest. The gap between CatBoost and the random forest (0.0078) is not large when compared with the fold-to-fold standard deviation of CatBoost (0.0052), so CV alone does not prove that CatBoost is better. It is the combination with the validation results that supports the choice.

Table 4.5 gives the validation results, where each model was fitted once on the fit set and scored on the held-out patients.

Table 4.5 Validation results (sorted by AP).

Model	ROC-AUC	AP
CatBoost	0.6779	0.2328
CatBoost + Class Weight	0.6690	0.2248
Random Forest	0.6753	0.2209
Logistic Regression	0.6653	0.2132
Decision Tree	0.6483	0.2122
Logistic Regression + Class Weight	0.6671	0.2120
Random Forest + Class Weight	0.6701	0.2071
Decision Tree + Class Weight	0.6437	0.2052

The ranking is similar to the cross-validation ranking. CatBoost is again first in both AP (0.2328) and ROC-AUC (0.6779), and the unweighted versions are ahead of the weighted ones in all four families. Below the first place the order changes a little: the validation AP of the weighted CatBoost moves ahead of the random forest, and the plain decision tree overtakes the weighted logistic regression. These shifts are small, and they are consistent with the modest differences seen in cross-validation. Overall, ROC-AUC values for all models lie in a narrow band between 0.644 and 0.678, which confirms the

weak-signal setting described in Section 1.2.4: no model separates the classes sharply, and the choice of model changes AP by a few hundredths. CatBoost was selected for tuning because it was first on both criteria.

4.3.2 Final model performance on the test set

Protocol. The tuned CatBoost model (depth 5, 764 iterations, learning rate 0.0582, L2 leaf regularisation 3.31) was refitted on the full training set of 79,541 encounters and applied once to the held-out test set of 19,802 encounters (13,998 patients; 2,216 positives, 17,586 negatives). The decision threshold, 0.1450, was computed from the validation predictions of the model trained on the fit split (63,807 encounters) and was not changed after seeing test results.

Table 4.6 Headline test performance with 95% patient-cluster bootstrap intervals (2,000 replicates).

Metric	Estimate	95% CI
ROC-AUC	0.677	0.663 – 0.691
Average Precision (AP)	0.244	0.221 – 0.268
Recall	0.431	0.404 – 0.458
Precision	0.217	0.201 – 0.232
F_1	0.288	0.270 – 0.306
Specificity	0.804	0.796 – 0.811

Further point estimates at the same threshold: accuracy 0.7622, negative predictive value 0.918, MCC 0.178, F_2 0.360, Brier score 0.0936, log loss 0.3272, and 22.2% of encounters flagged.

Table 4.7 Confusion matrix at threshold 0.1450 (Figure 4.10).

	Predicted 0	Predicted 1	Total
Actual 0	TN = 14,139	FP = 3,447	17,586
Actual 1	FN = 1,262	TP = 954	2,216
Total	15,401	4,401	19,802

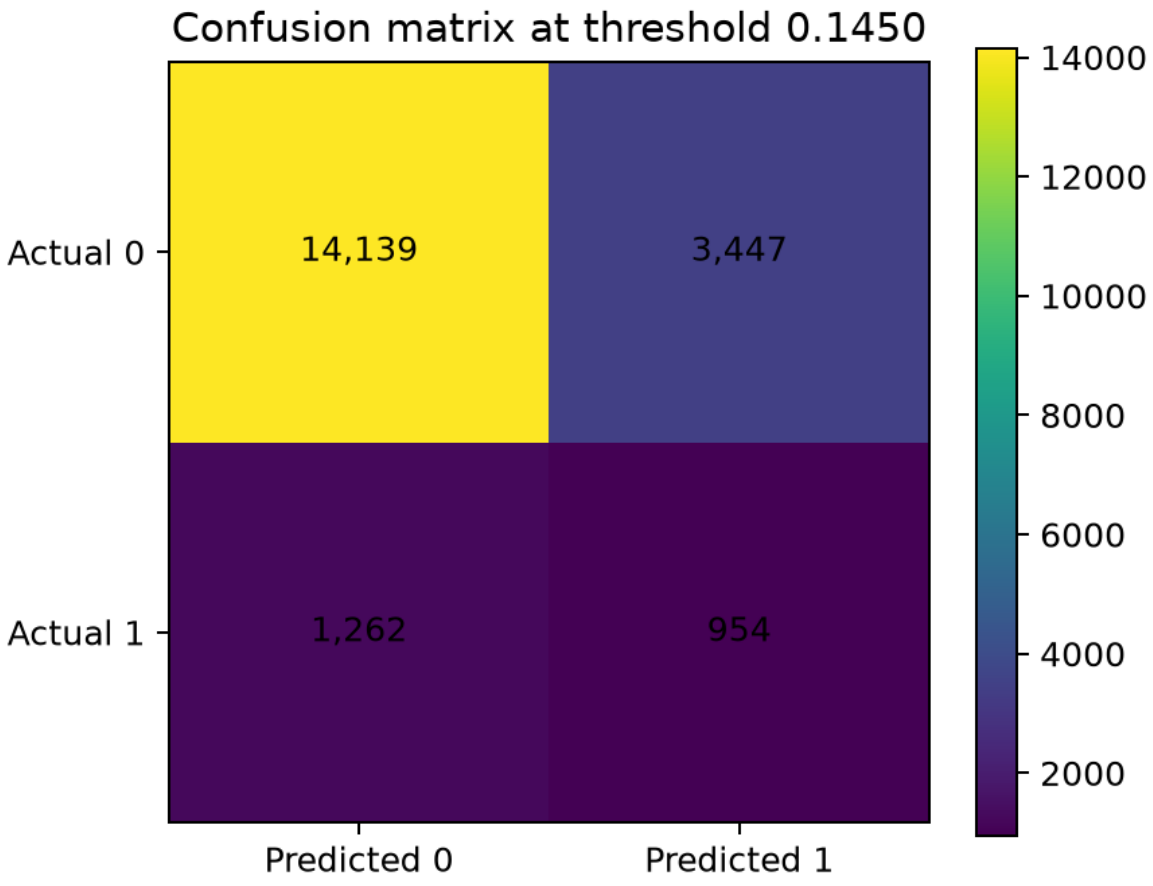


Figure 4.10 Confusion matrix at the selected threshold.

Interpretation. The ranking quality is clearly better than chance but modest. The ROC-AUC interval (0.663–0.691) excludes 0.5, and the AP interval (0.221–0.268) lies entirely above the prevalence of 0.112; the point estimate of AP is 2.18 times the random level. Precision at the operating point is 0.217, i.e. 1.94 times the base rate (lift), and the negative predictive value of 0.918 is only slightly above the 0.888 expected by chance, which shows that a "not flagged" result carries little reassurance. An MCC of 0.178 confirms a weak overall association between prediction and outcome.

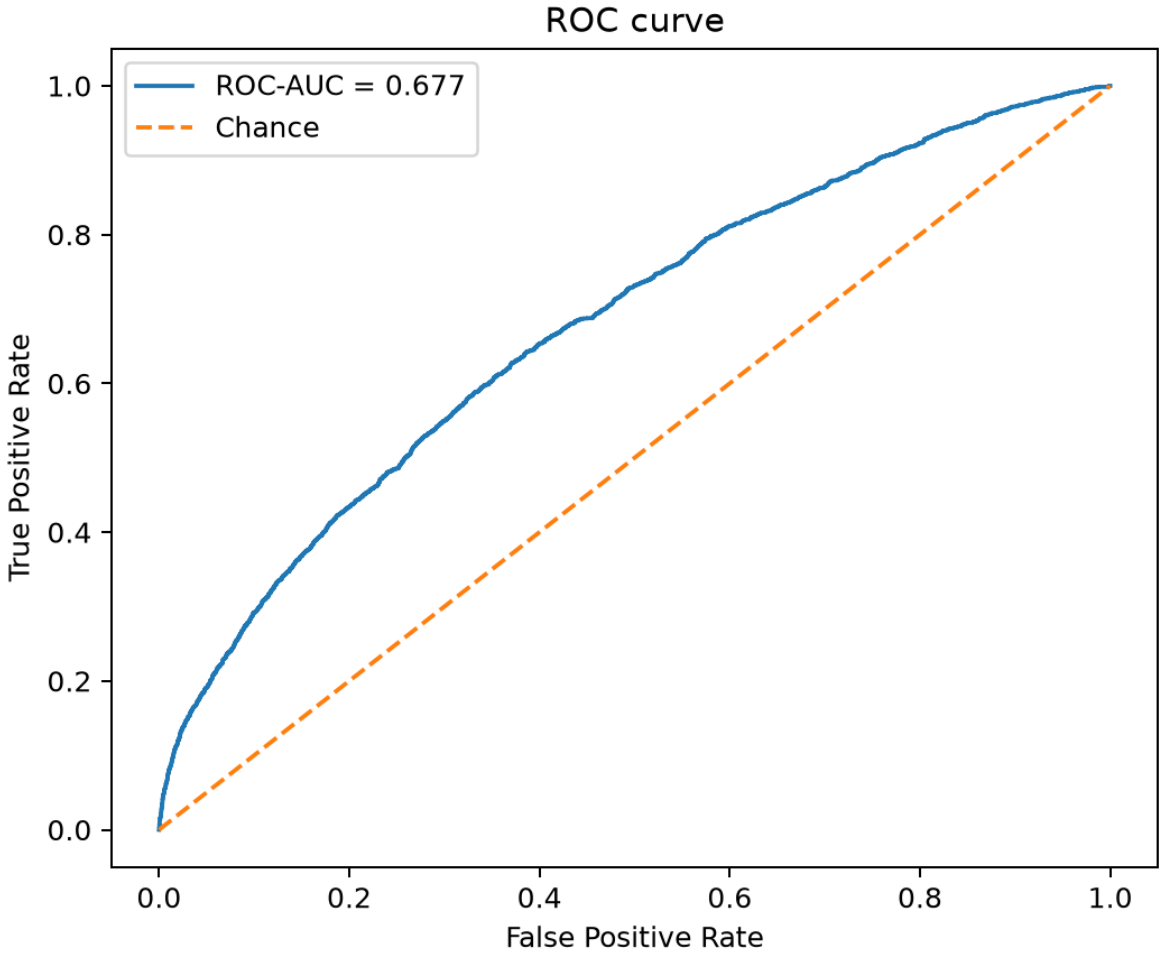


Figure 4.11 ROC Curve.

In terms of errors, the model finds 954 of 2,216 readmissions (43.1%) and misses 1,262 (57.0%). It raises 3,447 false alarms, which is 19.6% of all non-readmitted encounters ($1 - \text{specificity} = 0.804$). The confidence intervals are narrow enough (about ± 0.027 for recall and ± 0.015 for precision) that these operating-point figures are stable across patient resamples, but they describe this test set only.

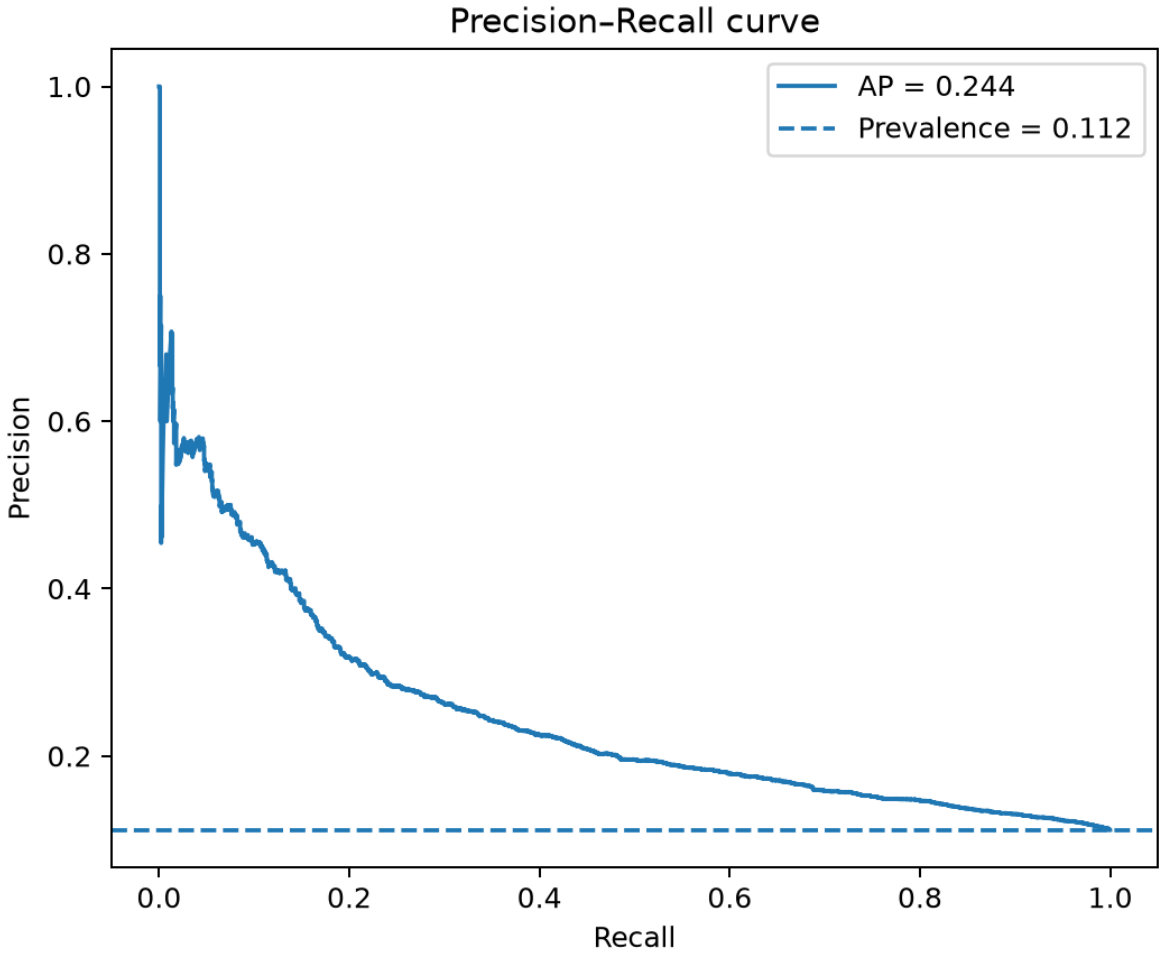


Figure 4.12 Precision–Recall curve with the prevalence baseline.

Table 4.8 Trivial reference policies on the same test set.

Policy	Accuracy	Recall	Precision	F_1	F_2	AP	ROC-AUC
Flag nobody	0.8881	0.000	undefined	0.000	0.000	0.1119	0.500
Flag everybody	0.1119	1.000	0.1119	0.2013	0.3865	0.1119	0.500
Tuned CatBoost ($t = 0.1450$)	0.7622	0.4305	0.2168	0.2883	0.3596	0.2444	0.6773

Two points deserve attention. First, the model beats both references on F_1 (0.288 against 0.201) and, unlike them, on AP and ROC-AUC. Second, flagging everybody reaches a higher F_2 (0.387) than the model (0.360), because F_2 rewards recall so strongly that a trivial all-positive policy scores well. This is a warning against using recall-heavy summary scores in isolation; whether the model is useful must be judged with explicit costs (Section 4.5).

4.3.3 Comparison with baselines and statistical significance

All nine models (four algorithms with and without class weights, plus the tuned CatBoost) were scored on the same ordered test encounters, each at its own validation-selected threshold. Rows are sorted by AP.

Table 4.9 Candidate models on the common test set.

Model	Threshold	AP	ROC-AUC	Recall	Precision	F_1	Specificity	Flagged %
CatBoost (default)	0.1279	0.2449	0.6759	0.5036	0.1952	0.2814	0.7384	28.9
Tuned CatBoost (final)	0.1450	0.2444	0.6773	0.4305	0.2168	0.2883	0.8040	22.2
CatBoost + class weight	0.5475	0.2307	0.6742	0.4580	0.2092	0.2873	0.7819	24.5
Random Forest	0.1281	0.2240	0.6676	0.5126	0.1926	0.2800	0.7292	29.8
Logistic Regression	0.1271	0.2178	0.6666	0.4901	0.1963	0.2803	0.7472	27.9
LR + class weight	0.5442	0.2167	0.6679	0.4616	0.1995	0.2786	0.7666	25.9
RF + class weight	0.5330	0.2154	0.6631	0.4558	0.1949	0.2731	0.7628	26.2
Decision Tree	0.1351	0.2051	0.6496	0.4517	0.1907	0.2681	0.7584	26.5
DT + class weight	0.6174	0.1974	0.6452	0.2617	0.2272	0.2432	0.8878	12.9

To decide whether the differences are real, each competitor was compared with the tuned CatBoost on the same 2,000 patient-cluster bootstrap samples. Differences are defined as tuned CatBoost minus the competitor, so positive values favour the final model. Holm-adjusted p-values are over the 8 comparisons; 0.008 is the smallest value the procedure can return with 2,000 replicates.

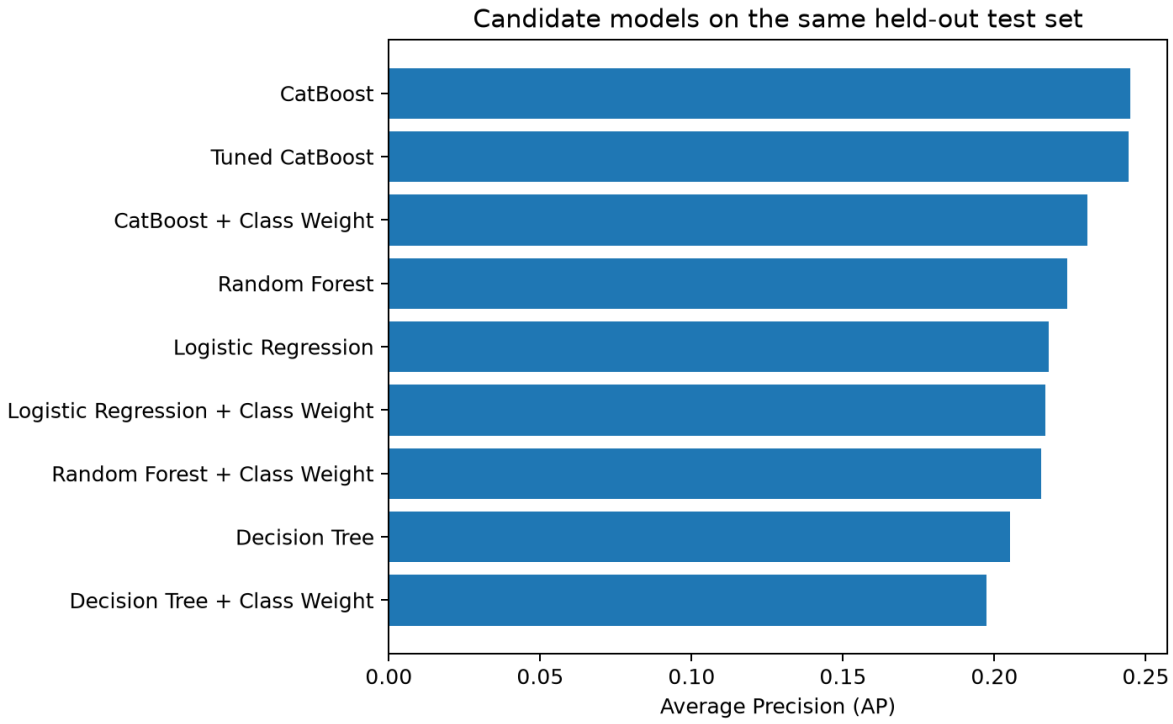


Figure 4.13 Average Precision (AP) comparison across models.

Table 4.10 Paired differences in the ranking metrics and F_1 (tuned CatBoost minus competitor).

Competitor	ΔROC-AUC [95% CI]	p (Holm)	ΔAP [95% CI]	p (Holm)	ΔF_1 [95% CI]	p (Holm)
Logistic Regression	$+0.0107$ $[+0.0055, +0.0159]$	0.008	$+0.0266$ $[+0.0169, +0.0355]$	0.008	$+0.0080$ $[-0.0006, +0.0167]$	0.210
Decision Tree	$+0.0278$ $[+0.0195, +0.0363]$	0.008	$+0.0393$ $[+0.0288, +0.0500]$	0.008	$+0.0202$ $[+0.0099, +0.0301]$	0.008
Random Forest	$+0.0098$ $[+0.0051, +0.0144]$	0.008	$+0.0204$ $[+0.0120, +0.0303]$	0.008	$+0.0084$ $[+0.0001, +0.0162]$	0.192
CatBoost (default)	$+0.0014$ $[-0.0015, +0.0043]$	0.355	-0.0005 $[-0.0047, +0.0040]$	0.875	$+0.0070$ $[-0.0005, +0.0141]$	0.210
LR + class weight	$+0.0094$ $[+0.0045, +0.0145]$	0.008	$+0.0277$ $[+0.0178, +0.0368]$	0.008	$+0.0097$ $[+0.0015, +0.0176]$	0.120
DT + class weight	$+0.0321$ $[+0.0236, +0.0406]$	0.008	$+0.0470$ $[+0.0360, +0.0577]$	0.008	$+0.0451$ $[+0.0309, +0.0598]$	0.008
RF + class weight	$+0.0143$ $[+0.0096, +0.0190]$	0.008	$+0.0290$ $[+0.0188, +0.0394]$	0.008	$+0.0153$ $[+0.0065, +0.0235]$	0.012
CatBoost + class weight	$+0.0031$ $[-0.0009, +0.0070]$	0.246	$+0.0137$ $[+0.0071, +0.0205]$	0.008	$+0.0011$ $[-0.0059, +0.0079]$	0.773

Table 4.11 Paired differences in the operating-point metrics at each model's own threshold.

Competitor	ΔRecall	p (Holm)	ΔSpecificity	p (Holm)
Logistic Regression	-0.0596	0.008	$+0.0568$	0.008
Decision Tree	-0.0212	0.019	$+0.0456$	0.008
Random Forest	-0.0821	0.008	$+0.0748$	0.008
CatBoost (default)	-0.0731	0.008	$+0.0656$	0.008
LR + class weight	-0.0311	0.008	$+0.0374$	0.008
DT + class weight	$+0.1688$	0.008	-0.0838	0.008
RF + class weight	-0.0253	0.008	$+0.0412$	0.008
CatBoost + class weight	-0.0275	0.008	$+0.0221$	0.008

Reading the results.

- **Ranking quality (AP, ROC-AUC).** The final model is significantly better than logistic regression, decision tree, random forest and all class-weighted variants of these, since every confidence interval for ΔAP excludes zero. The AP gain over logistic regression is 0.027 (about 12% relative), over random forest 0.020, and over the decision tree 0.039. The gain is small in absolute terms, which matches the weak-signal finding of Section 1.2.4, but it is statistically reliable.

- **Gradient boosting with and without tuning.** Tuned CatBoost and default CatBoost are statistically indistinguishable in AP ($\Delta = -0.0005, p = 0.875$) and ROC-AUC ($\Delta = +0.0014, p = 0.355$). The extra tuning therefore did not measurably improve ranking on the test set; the benefit of the tuned model lies in the operating point it yields (Section 4.4.3), not in its AUC or AP.

- **Class weighting.** Class-weighted variants do not improve AP for any algorithm (CatBoost 0.2307 against 0.2449, random forest 0.2154 against 0.2240, decision tree 0.1974 against 0.2051; logistic regression is essentially unchanged, 0.2167 against 0.2178). Weighting mainly shifts the probability scale: thresholds for weighted models are around 0.53–0.62 whereas unweighted models use about 0.13, so weighted probabilities no longer estimate real risk.

- **Operating-point metrics.** Differences in F_1 are mostly not significant after Holm correction (logistic regression, random forest, default CatBoost and CatBoost with class weight all have intervals compatible with zero), because the F_1-optimal threshold produces a flat optimum. Recall and specificity differ significantly, but they mainly reflect different thresholds: the final model trades recall for specificity compared with most competitors, flagging 22.2% of encounters versus 25–30%. The one exception is the weighted decision tree, which has the highest specificity (0.888) but a recall of only 0.262.

Caveats. Operating-point metrics of different models are not directly comparable because each model uses its own validation-selected threshold; only AP and ROC-AUC are threshold-free. Validation metrics at threshold 0.5 in notebook 02 are descriptive and must not be compared with test metrics at tuned thresholds. All comparisons rest on one patient-level train/test split.

4.3.4 Comparison with published results

Numbers from other studies are not directly comparable with Table 4.6 unless the split unit, exclusions, target definition and metric are identical. Table 4.12 therefore

records what could be verified from the sources and the protocol information that is given.

Table 4.12 Published results on the same dataset family and this work.

Study	Method	Reported metric	Split unit stated?
Sarthak et al. (2020), EmbPred30	DNN with categorical embeddings	AUROC 97.4%, accuracy 95.2%	Not stated in the abstract
Goudjerkan and Jayabalan (2019)	Multilayer perceptron with extensive preprocessing	AUC about 95%, recall about 99%	Not stated in the abstract
Emi-Johnson and Nkrumah (2025)	LR, RF, XGBoost, DNN	XGBoost reported best; exact AUC not verified from repository files	Not verified from full text in this project
Studies with a clean patient-level protocol	LR / tree-based models	No verified aggregate value available in repository	Not verified study by study
This work	**Tuned CatBoost**	**ROC-AUC** 0.677 (0.663–0.691)**; AP** 0.244 (0.221–0.268)	**Yes: patient-level split,** 0 **shared patients**

Discussion. The cited values above 0.9, if confirmed in the original papers, would be 0.25 to 0.30 ROC-AUC points higher than this work. The repository results show that the same family of administrative features yields ROC-AUC values around 0.65–0.68 across the tested algorithms, including logistic regression. However, the repository alone does not establish the split protocol of the cited external studies, nor does it support a verified like-for-like range for studies using patient-level splits. Encounter-level random splits can inflate apparent performance when repeat encounters from the same patient appear in both training and test sets. We therefore do not claim that this model is "worse" than the cited studies; we report that this work used a patient-level split and that the external comparisons remain provisional until the original papers are checked. The exact AUC and split protocol for Emi-Johnson and Nkrumah (2025), as well as values and protocols of other purported patient-level studies, are not verified by the repository outputs.

4.4 Ablation analysis

4.4.1 Effect of class weighting

Table 4.13 compares each model with its class-weighted counterpart, using the numbers from Tables 4.4 and 4.5.

Table 4.13 Effect of class weighting on AP.

Model family	CV AP (default)	CV AP (weighted)	Val AP (default)	Val AP (weighted)
Logistic Regression	0.2091	0.2073	0.2132	0.2120
Decision Tree	0.1920	0.1899	0.2122	0.2052
Random Forest	0.2192	0.2109	0.2209	0.2071
CatBoost	0.2270	0.2161	0.2328	0.2248

Class weighting did not improve AP for any of the four models, either in cross-validation or on the validation split. The loss is small for logistic regression (about 0.002 in CV AP and 0.001 on validation) and larger for the random forest and CatBoost (0.008 to 0.014). ROC-AUC shows the same direction for CatBoost, the random forest and the decision tree, while logistic regression is basically unchanged (0.6653 versus 0.6671).

A likely reason is that reweighting changes how much each example counts in training but adds no new information about who is readmitted. With a noisy signal, giving about four times more weight to a few thousand positive examples can make the flexible models fit the noise in those examples. Class weights are also not needed to reach a good operating point here, because the threshold in Section 3.5 already moves the decision boundary to where it is useful. This explanation is a hypothesis and was not tested separately. What the data do show is that, for this problem, the simpler unweighted models rank patients at least as well as the weighted ones, and the unweighted CatBoost was kept for the final model.

4.4.2 Effect of hyperparameter tuning

The default CatBoost (200 iterations, depth 6, learning rate 0.1) reached a mean AP of 0.2270 in 3-fold grouped cross-validation, and 0.2328 on the validation split. The best configuration found by the random search reached a mean AP of 0.2336 in 5-fold grouped cross-validation on the same fit set. The cross-validated AP therefore rose by about 0.007, which is a gain of roughly 3% in relative terms.

This comparison should be read with care, because the two cross-validation scores were computed with different numbers of folds (3 and 5), so the fold sizes and the exact splits differ. The size of the gain is also of the same order as the fold-to-fold standard deviation of the default model (0.0052). Our reading is that tuning gives a small and that

most of the performance of CatBoost on this dataset comes from the features and the model family and not from the hyperparameters.

4.4.3 Effect of the decision threshold

The threshold converts a risk score into a flag and does not change the ranking, so AP and ROC-AUC are identical for every threshold; only recall, precision, flagged share and F_1 change. Table 4.14 compares the selected operating point with alternatives that are available from the experiments.

Table 4.14 Operating points on the test set.

Operating point	Threshold	Flagged %	Recall	Precision	F_1	Specificity
Tuned CatBoost, max-F_1 on val (selected)	0.1450	22.2	0.4305	0.2168	0.2883	0.8040
Default CatBoost, its max-F_1 on val	0.1279	28.9	0.5036	0.1952	0.2814	0.7384
Tuned CatBoost at conventional cut-off 0.5	0.5000	0.36	0.0181	0.5556	0.0350	0.9982
Flag everybody	—	100	1.000	0.1119	0.2013	0.000
Flag nobody	—	0	0.000	undefined	0.000	1.000

The dashed line marks the validation-selected threshold; the curve is descriptive and is not used to choose a new threshold

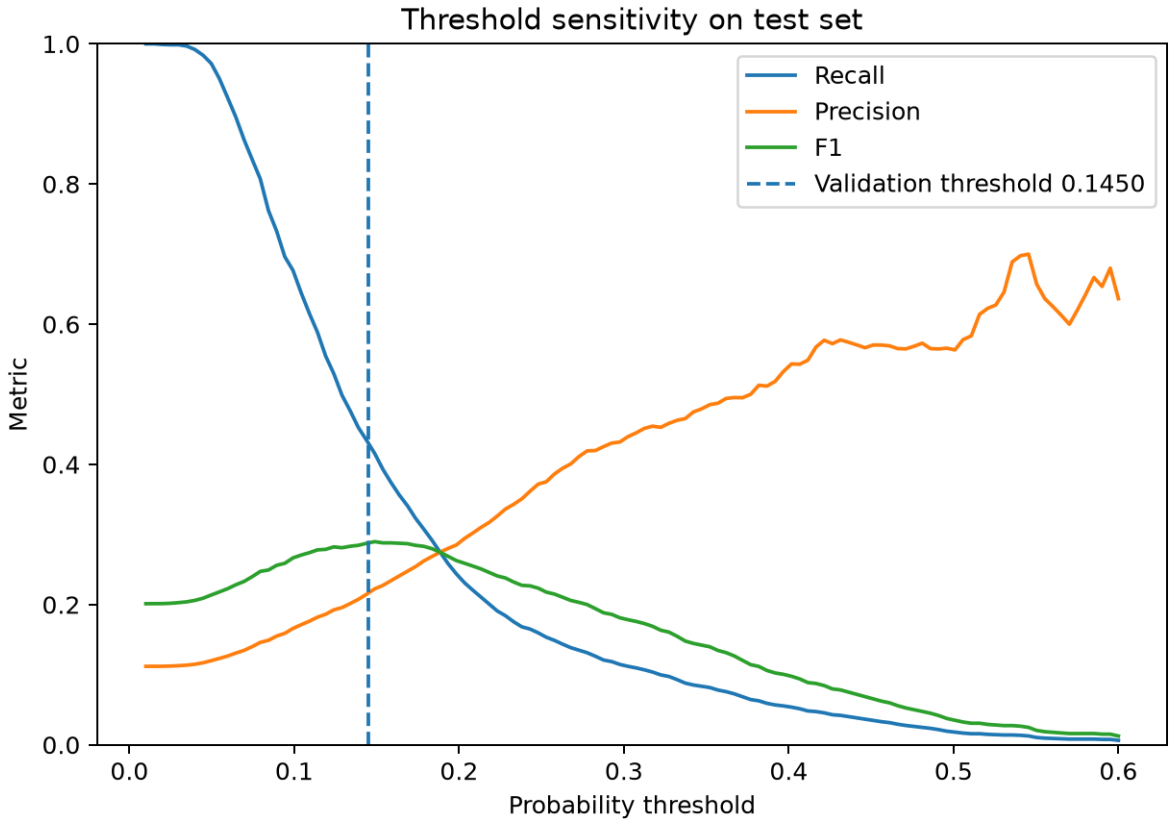


Figure 4.14 Recall, precision and F1 against the threshold on the test set.

Discussion. Moving from the selected point to the lower threshold of default CatBoost (a model with practically the same ranking, AP 0.2449 against 0.2444) raises the flagged share from 22.2% to 28.9% and recall from 0.431 to 0.504, i.e. 7.3 percentage points more readmissions are found, while precision drops from 0.217 to 0.195 and specificity from 0.804 to 0.738. F_1 changes by only 0.007, a difference that is not significant (Table 4.10), so the F_1 optimum is flat and the choice between these points is a business decision rather than a statistical one: a hospital with spare follow-up capacity would prefer the lower threshold, one with scarce capacity the higher. Using the conventional 0.5 cut-off is inappropriate because predicted risks of an unweighted model on a population with 11% prevalence rarely reach 0.5, so very few encounters would be flagged and recall would be close to zero; at threshold 0.5, the test set flags 0.36% of encounters, with recall 0.0181, precision 0.5556, F_1 0.0350 and specificity 0.9982. The weighted models show the same effect from the other side: class weights shift their scores so far upward that their F_1-optimal thresholds are 0.53–0.62.

4.5 Operational and cost-sensitive analysis

A model is only valuable if acting on it is cheaper or safer than not acting. This section evaluates the fixed operating point (threshold 0.1450) under explicit cost assumptions. The costs are hypothetical, since the dataset contains no cost data, and are expressed in units of one false alarm.

4.5.1 Cost ratio between false negatives and false positives

Let a false positive cost 1 unit and a false negative cost r units ($r > 1$ when a missed readmission is worse than an unnecessary intervention). The total cost of a policy is $r \cdot \mathrm{FN} + \mathrm{FP}$. For the three policies on the test set, with $\mathrm{TP} = 954$, $\mathrm{FP} = 3,447$, $\mathrm{FN} = 1,262$, $\mathrm{TN} = 14,139$ and $2,216$ positives:

- **Flag nobody:** cost = $r \times 2,216$.
- **Flag everybody:** cost = $17,586$ (all negatives become false alarms).
- **Model at $t = 0.1450$:** cost = $r \times 1,262 + 3,447$.

The model is cheaper than flagging nobody when $r > 3,447/954 = 3.61$, and cheaper than flagging everybody when $r < 14,139/1,262 = 11.20$. The model is therefore economically useful only for cost ratios between about 3.6 and 11.2.

Interpretation. At balanced costs (1 : 1 or 2 : 1) the best action is to intervene on nobody, because the model produces too many false alarms. Only when a missed readmission is at least about four times as costly as an unnecessary intervention does

Table 4.15 Total cost (in false-alarm units) for several cost ratios.

r (FN : FP)	Flag nobody	Flag everybody	Model ($t = 0.1450$)	Best policy	Model vs best trivial policy
1 : 1	2,216	17,586	4,709	Flag nobody	+112.5% (worse)
2 : 1	4,432	17,586	5,971	Flag nobody	+34.7% (worse)
3 : 1	6,648	17,586	7,233	Flag nobody	+8.8% (worse)
5 : 1	11,080	17,586	9,757	Model	−11.9% (saving)
10 : 1	22,160	17,586	16,067	Model	−8.6% (saving)
15 : 1	33,240	17,586	22,377	Flag everybody	+27.2% (worse)

the model pay off, and the saving is modest (about 9–12% in this range). At very high ratios, intervening on everyone becomes cheaper, because the model's recall of 43% still leaves many missed cases. The threshold chosen by max-F_1 corresponds to an implied cost ratio of about $1/0.145 - 1 = 5.9$ for a well-calibrated model, which lies inside the useful band.

The analysis above holds the threshold fixed. Under ideal calibration and specified costs, the theoretical cost-minimising threshold is $t^* = 1/(1+r)$, i.e. 0.25 for $r = 3$, 0.167 for $r = 5$ and 0.091 for $r = 10$. Empirical minimisation of $r \cdot \mathrm{FN} + \mathrm{FP}$ on the saved validation predictions gives the operating points below. These thresholds are selected on validation data only; their costs are descriptive estimates on that validation split, not unbiased test estimates.

Table 4.16 Validation-set thresholds minimising empirical cost.

FN:FP cost ratio	Optimal threshold	Flagged (%)	FN	FP	Model cost	Flag nobody	Flag everybody	Cost saving vs best trivial policy
3:1	0.2731	3.58	1,586	348	5,106	5,406	13,932	5.5%
5:1	0.1833	11.87	1,296	1,362	7,842	9,010	13,932	13.0%
10:1	0.0858	56.01	422	7,432	11,652	18,020	13,932	16.4%

4.5.2 Decision-curve analysis

At the selected threshold probability $p_t = 0.1450$, the odds weight is $p_t/(1-p_t) = 0.170$. The net benefit of the model is $954/19,802 - (3,447/19,802) \times 0.170 = 0.0187$. Treating everyone gives $0.1119 - 0.8881 \times 0.170 = -0.0387$, and treating no one gives 0. The model is therefore better than both references at this threshold: 0.0187 corresponds to about 19 net true positives per $1,000$ encounters after charging each false positive at its odds-weighted cost.

4.5.3 Recall at top-k, gain and lift

If only a fixed share k of discharged patients can receive follow-up, the relevant quantity is recall among the k% highest-risk encounters. At the selected operating point, 22.2% of encounters are flagged and recall is 43.05%; the top-k table below gives the corresponding results for fixed capacity limits. Gain is Recall@k divided by k, and lift

is Precision@k divided by the test-set prevalence (0.1119).

Table 4.17 Recall@k, gain and lift on the held-out test set.

Top-risk share (k)	Encounters selected	True readmissions captured	Recall@k	Gain	Lift
5%	990	362	0.1634	3.267	3.267
10%	1,980	561	0.2532	2.532	2.532
20%	3,960	889	0.4012	2.006	2.006
30%	5,941	1,153	0.5203	1.734	1.734
50%	9,901	1,564	0.7058	1.412	1.412

The top 5% of encounters captures 16.3% of readmissions, while the top 20% captures 40.1%. This ranking-based analysis is useful when follow-up capacity is fixed; it does not imply that the predicted probabilities are accurate for individual patients.

4.6 Error, calibration and subgroup analysis

4.6.1 Error analysis

At the selected threshold the model makes $1,262$ false negatives and $3,447$ false positives. Reading the subgroup table (4.6.3) together with the confusion matrix shows where the misses occur. Encounters discharged home (code 1) form the largest group ($12,030$ encounters, $1,081$ readmissions, prevalence 9.0%), and recall within it is only 0.305, which corresponds to 751 of the $1,262$ false negatives (59.5%). This is partly a size effect: home discharges make up 60.8% of test encounters but 48.8% of positives.

Among the groups large enough to interpret, the share of encounters flagged as positive rises with the group's prevalence: about 12.7% for home (prevalence 9.0%), 28.5% for home health (12.3%) and 40.1% for skilled nursing (14.4%), against 22.2% overall. Recall therefore rises with prevalence (0.305, 0.442, 0.503), while within-group ROC-AUC stays modest (0.661, 0.642, 0.581). This pattern is consistent with the model using discharge disposition as a coarse risk marker (the strongest categorical association in the EDA, Cramér's $V = 0.112$) and having limited ability to separate high-risk from low-risk patients within the large home-discharge group. No feature-importance or ablation analysis was run, so this remains an interpretation rather than a tested finding. A few small disposition groups (for example codes 22 and 5, with 405 and 241 encounters) show very high recall, but they fall below the 500-encounter reporting threshold and are not used as evidence here.

The error labels for every test encounter (TP/FN/FP/TN) are stored in `outputs/evaluation/test_error_labels.csv`. A feature-distribution comparison between false negatives and true positives, and between false positives and true negatives, has not been computed in the saved outputs; no further feature-level error claims

are made here.

4.6.2 Calibration

The final model is trained with log loss and without class weights or resampling, so its scores are on the natural risk scale; this is why the F_1-optimal threshold (0.145) lies close to the prevalence (0.112), whereas weighted models need thresholds of 0.53–0.62. Summary measures on the test set:

Measure	**Final model**	**Constant prediction at prevalence**	**Relative improvement**
Brier score	0.0936	0.0994	5.8% lower
Log loss	0.3272	0.3505	6.7% lower

Both scores are better than a constant forecast, so the probabilities contain real information, but the margin is small, which again reflects the weak signal. Figure 4.15 shows the reliability curve with 10 quantile bins. The curve is reasonably close to the diagonal overall; in the highest-risk bin, the mean predicted probability is 0.2803 and the observed positive fraction is 0.2832. ECE, computed as the bin-size-weighted absolute difference between observed and mean predicted risk over the same 10 quantile bins, is 0.0063. A logistic recalibration of the observed test outcomes on the logit of predicted risk gives a slope of 1.033 and intercept 0.048; this is descriptive on the test set, not a fitted post-hoc calibration model for deployment. Calibration is descriptive here and does not change the threshold.

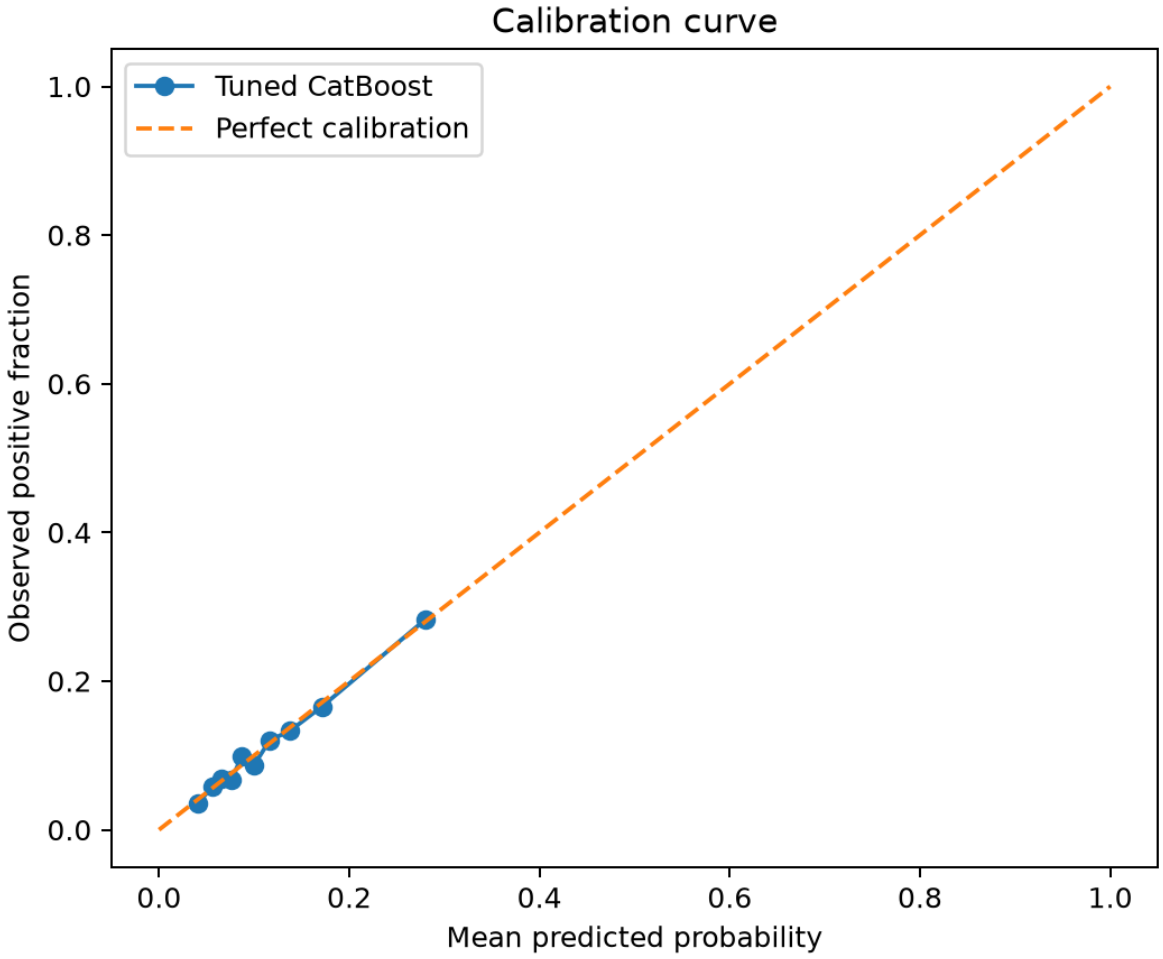


Figure 4.15 Calibration Curve of the Tuned CatBoost Model.

4.6.3 Subgroup performance

Metrics are computed at the single global threshold of 0.1450. Groups with fewer than 500 encounters or fewer than 50 positives are marked as unstable and should not be interpreted. Table 4.18 shows the larger subgroups (full results in `outputs/evaluation/tables/07_subgroup_performance.csv`).

Findings.

- **Age.** ROC-AUC is highest in the middle-aged groups (0.736 at 30–40, 0.711 at 50–60) and declines steadily with age to 0.642 at 80–90 and 0.524 at 90–100, where the model is close to random; the 90–100 estimate rests on only 70 positives. Groups under 30 years ($n = 29,138,388$) are too small to interpret even though some metrics look high (for example ROC-AUC 0.873 at 20–30).

- **Gender.** Female recall is 5.9 percentage points higher than male recall (0.458 versus 0.399; 95% CI for the difference $[1.8, 10.0]$, $p \approx 0.005$, normal approximation on encounter-level counts), at nearly identical precision (0.218 versus 0.215). The

Table 4.18 Performance by subgroup on the test set.

Feature	Group	n	Positives	Prevalence	AP	ROC-AUC	Recall	Precision
Age	30–40	729	77	0.106	0.334	0.736	0.506	0.255
Age	40–50	1,909	202	0.106	0.212	0.673	0.371	0.214
Age	50–60	3,469	356	0.103	0.276	0.711	0.416	0.273
Age	60–70	4,266	464	0.109	0.225	0.659	0.388	0.209
Age	70–80	5,040	587	0.116	0.226	0.666	0.450	0.207
Age	80–90	3,287	390	0.119	0.237	0.642	0.438	0.184
Age	90–100	547	70	0.128	0.188	0.524	0.357	0.166
Gender	Female	10,645	1,188	0.112	0.246	0.685	0.458	0.218
Gender	Male	9,157	1,028	0.112	0.243	0.669	0.399	0.215
Race	Caucasian	14,886	1,707	0.115	0.252	0.675	0.433	0.222
Race	African American	3,707	396	0.107	0.205	0.672	0.432	0.195
Primary diagnosis	Circulatory	5,942	691	0.116	0.208	0.632	0.385	0.197
Primary diagnosis	Respiratory	2,709	260	0.096	0.242	0.698	0.404	0.226
Primary diagnosis	Diabetes	1,731	227	0.131	0.311	0.723	0.590	0.261
Primary diagnosis	Digestive	1,846	215	0.116	0.241	0.663	0.400	0.248
Primary diagnosis	Injury	1,378	166	0.120	0.283	0.701	0.536	0.216
Primary diagnosis	Genitourinary	1,002	99	0.099	0.168	0.609	0.323	0.142
Primary diagnosis	Musculoskeletal	963	86	0.089	0.276	0.756	0.465	0.221
Primary diagnosis	Neoplasms	615	68	0.111	0.156	0.592	0.235	0.150
Discharge	Home (1)	12,030	1,081	0.090	0.206	0.661	0.305	0.215
Discharge	Skilled nursing (3)	2,731	394	0.144	0.192	0.581	0.503	0.181
Discharge	Home health (6)	2,593	319	0.123	0.222	0.642	0.442	0.191
Discharge	Unknown	934	112	0.120	0.202	0.642	0.330	0.190

interval ignores repeated encounters per patient and is therefore somewhat optimistic; with a design effect of 1.3 it becomes $[1.2, 10.6]$. The gap is unlikely to be due to chance alone, but its cause and clinical relevance are not assessed. The ROC-AUC difference (0.685 versus 0.669) was not tested.

- **Race.** Recall is practically identical for Caucasian and African American patients (0.433 and 0.432; difference 0.1 points, 95% CI $[-5.3, +5.5]$, encounter-level). Precision (0.222 versus 0.195) and AP (0.252 versus 0.205) differ, but these differences were not tested. Other racial groups have fewer than 500 encounters and are not interpretable.

- **Primary diagnosis.** The model works best when the primary diagnosis is diabetes (ROC-AUC 0.723, recall 0.590) or musculoskeletal (0.756) and worst for neoplasms (0.592), genitourinary (0.609) and circulatory conditions (0.632). Circulatory diagnoses are the largest group (30% of the test set), so the weak performance there pulls down the overall result.

- **Discharge disposition.** The within-group ROC-AUC is weak for patients discharged to skilled nursing facilities (0.581) although their recall is high (0.503)

because their prevalence is high (14.4%); the model flags them largely on the basis of the group itself.

These differences are exploratory. Confidence intervals for the gender and race recall differences were computed from encounter-level counts and do not account for repeated encounters per patient; no intervals or tests were computed for the other subgroup metrics (AP, ROC-AUC, precision). Because many subgroups are small and prevalence differs between them, raw differences should not be read as evidence of bias or clinical effects. Patient-cluster bootstrap intervals remain future work.

4.7 Chapter 4 conclusion

The experiments support four conclusions. First, the final tuned CatBoost model reaches an AP of 0.244 (CI 0.221–0.268) and a ROC-AUC of 0.677 (CI 0.663–0.691) on a patient-level held-out test set; this is statistically better than all other candidates in AP, and in ROC-AUC as well, except for untuned CatBoost and class-weighted CatBoost, from which it cannot be distinguished. Second, at the max-F_1 threshold of 0.1450 the model finds 43.1% of readmissions while flagging 22.2% of encounters (precision 0.217, lift 1.94), and it is worth deploying only if a missed readmission costs roughly 3.6 to 11.2 times as much as a false alarm. Third, the probabilities are better than a constant forecast but only slightly so, and performance is uneven across age, diagnosis and discharge groups, with the model leaning heavily on discharge disposition. Fourth, published results far above 0.9 AUC should not be compared with ours without checking the evaluation protocol. Overall, the evidence indicates a useful but limited tool for prioritising follow-up resources.

CONCLUSION

General conclusion

This project developed a patient-level, leakage-aware pipeline for predicting 30-day readmission of diabetic inpatients on the Diabetes 130-US Hospitals dataset. After removing death and hospice encounters, the data (99,343 encounters, about 11% positive) were split by patient, preprocessed with ICD-9 grouping and explicit missing-value handling, enriched with seven utilisation features, and used to compare eight candidate models under patient-grouped cross-validation with AP as the primary metric. A tuned CatBoost model with a validation-selected threshold of 0.1450 was evaluated once on the untouched test set.

The main quantitative results are: AP 0.244 (2.18 times the random level of 0.112), ROC-AUC 0.677, recall 0.431, precision 0.217, F_1 0.288, specificity 0.804, MCC 0.178, with 22.2% of encounters flagged. Patient-cluster bootstrap with Holm correction showed that the model significantly outperforms all baselines except untuned CatBoost and CatBoost with class weights in AUC. Class weighting did not improve ranking, and tuning did not improve it significantly. The model beats trivial policies only for cost ratios of about 3.6 to 11.2, has net benefit 0.0187 at the selected threshold, and shows uneven subgroup performance.

The central message is that the information available in routinely collected administrative and clinical-process variables supports only weak-to-moderate risk stratification, and that strict evaluation reduces the apparently high published numbers to this level. The model can help prioritise limited follow-up resources but cannot be used to make individual-level decisions.

Limitations and future work

Limitations

- The strongest numerical correlation with the target is 0.167, while the strongest categorical association is Cramér's V = 0.112. Individual features therefore provide limited predictive information on their own.

- Correlation and Cramér's V examine individual associations rather than joint or causal relationships. Age and diagnosis plots also show group counts rather than readmission rates.

- Winsorization limits the influence of extreme values but reduces information about their magnitude. Broad ICD-9 grouping also sacrifices clinical detail, while missing-

value labels cannot recover the original values.

- One dataset, one patient-level train/test split and one random seed; the confidence intervals capture test-set sampling noise but not variation due to the split.

- The data are from 130 US hospitals in 1999–2008; practice patterns, coding (ICD-9) and diabetes care have changed, and there is no external or temporal validation.

- The threshold was computed from the validation predictions of the model trained on the fit split (63,807 encounters) and applied to the model refitted on the full training set (79,541 encounters), which can shift the operating point slightly.

- Limited categorical feature representation: Categorical variables were one-hot encoded before being passed to CatBoost, meaning its native categorical handling was not fully utilised.

- Limited hyperparameter search: The search covered only 20 random configurations across four parameters, and the tuned CatBoost model did not significantly outperform the default model on the test set.

- Cost ratios in Section 4.5 are assumptions; the dataset contains no intervention or penalty costs.

- Subgroup results are descriptive, unadjusted and unstable for small groups; the feature-engineering ablation and any SHAP analysis were not performed.

- Published results in Section 4.3.4 are not directly comparable, and some of their values still have to be verified against the original papers.

- Modeling was run in Google Colab and evaluation on a local machine (Python 3.13.9, scikit-learn 1.9.1, CatBoost 1.2.10), so small numerical differences are possible across environments.

Future work

- Improved EDA. Examine readmission rates with confidence intervals and explore multivariate and non-linear relationships.

- Compare different winsorization limits, ICD-9 grouping strategies and near-constant thresholds to assess their effects on model performance.

- Evaluate with repeated patient-grouped cross-validation or nested validation to include split variability in the uncertainty.
- Improve categorical feature handling: Use CatBoost's native categorical feature support instead of one-hot encoding.
- Explore alternative models: Evaluate LightGBM, XGBoost and ensemble methods to compare their performance with CatBoost.
- Enhance feature engineering: Test finer ICD-9 groupings and medication-change features to capture more detailed clinical information.
- Calibrate the scores (Platt or isotonic regression), report ECE, and select the threshold by minimising expected cost for a hospital-specific cost ratio.
- Produce the full decision curve, Recall@top-k and subgroup confidence intervals, and examine fairness-aware thresholds if differences persist.
- Add richer features such as longer utilisation history, laboratory values and clinical notes, and validate on a more recent or external dataset.
- Add SHAP-based interpretation and a feature-engineering ablation with identical patient-grouped splits.

REFERENCES

[1] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "Catboost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[3] L. Breiman, "Bagging predictors," *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996.

[4] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[5] J. Clore, K. Cios, J. DeShazo, and B. Strack, "Diabetes 130-us hospitals for years 1999–2008," UCI Machine Learning Repository, 2014.

[6] B. Strack, J. P. DeShazo, C. Gennings, J. L. Olmo, S. Ventura, K. J. Cios, and J. N. Clore, "Impact of hba1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records," *BioMed Research International*, vol. 2014, p. 781670, 2014.

[7] Centers for Medicare & Medicaid Services, "Hospital readmissions reduction program (hrrp)," U.S. Department of Health and Human Services, 2026. [Online]. Available: https://www.cms.gov/medicare/quality/value-based-programs/hospital-readmissions

[8] A. Artetxe, A. Beristain, and M. Graña, "Predictive models for hospital readmission risk: A systematic review of methods," *Computer Methods and Programs in Biomedicine*, vol. 164, pp. 49–64, 2018.

[9] E. R. Pfoh *et al.*, "Predicting and preventing acute care re-utilization by patients with diabetes," *Current Diabetes Reports*, vol. 21, no. 9, p. 34, 2021.

[10] Y. Shang, K. Jiang, L. Wang *et al.*, "The 30-days hospital readmission risk in diabetic patients: Predictive modeling with machine learning classifiers," *BMC Medical Informatics and Decision Making*, vol. 21, no. 1, p. 57, 2021.

[11] C. Neto, F. Senra, J. Leite, N. Rei, R. Rodrigues, D. Ferreira, and J. Machado, "Different scenarios for the prediction of hospital readmission of diabetic patients," *Journal of Medical Systems*, vol. 45, no. 1, p. 11, 2021.

[12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[13] scikit-learn developers, "sklearn.model_selection.groupshufflesplit," Scikit-learn documentation, 2024. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html

[14] ——, "3.1. cross-validation: Evaluating estimator performance," Scikit-learn User Guide, 2024. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html

[15] ——, "sklearn.metrics.average_precision_score," Scikit-learn documentation, 2024. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.average_precision_score.html

[16] ——, "sklearn.metrics.precision_score," Scikit-learn documentation, 2024. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html

[17] A. Agresti, *Categorical Data Analysis*, 3rd ed. Wiley, 2013.

[18] W. Bergsma, "A bias-correction for cramér's v and tschuprow's t," *Journal of the Korean Statistical Society*, vol. 42, no. 3, pp. 323–328, 2013.

[19] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.

[20] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.